Fundamentals of Neural Networks

Lecturer: Dr. Samuel Kakuba

February 17 and 19, 2026

Part of this material came from Prof. Jaeil Kim's Lecture Materials at the School of Computer Science and Engineering

Kyungpook National University

Index

01 Performance Measure & DL

02 Loss Functions

03 Optimization

04 Computational Graph & Backprop.

05 Hands-on: MLP using PyTorch

0

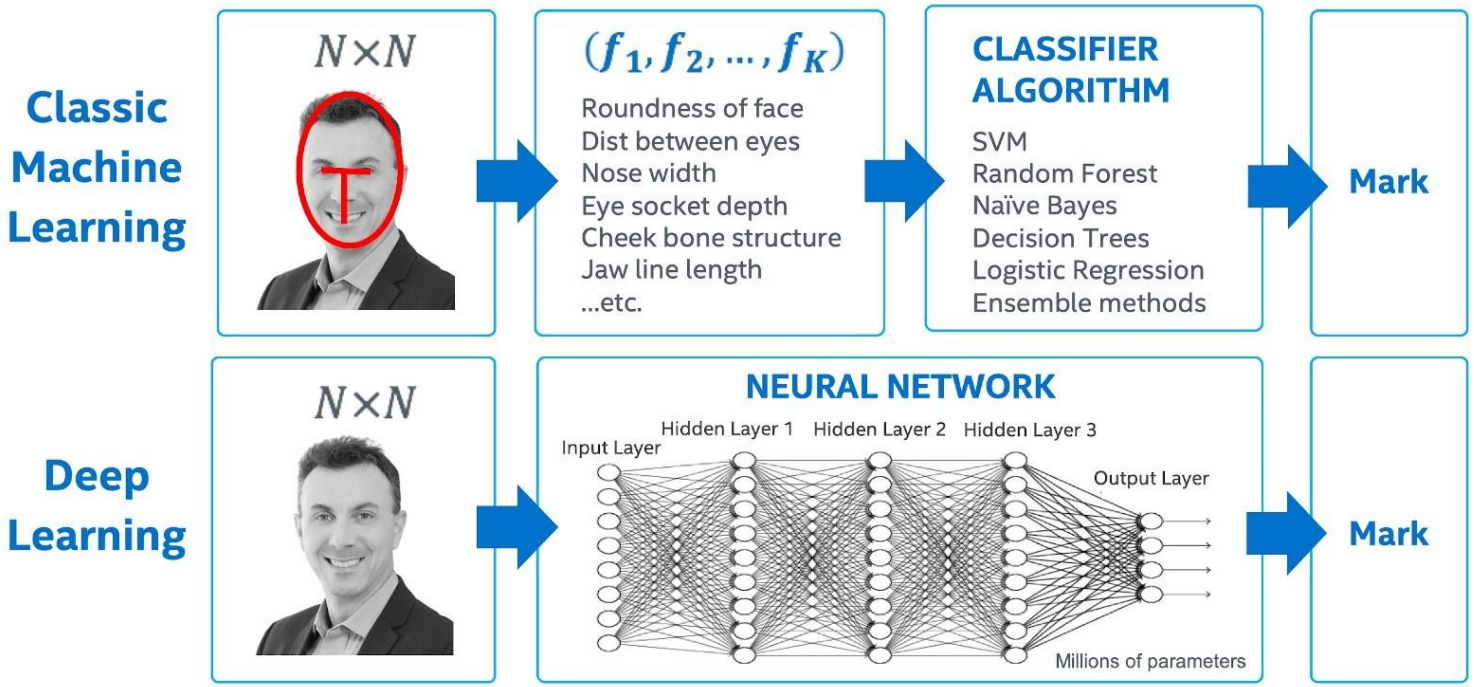
Objectives

- Understand the key components of machine learning
  - Task definition
  - Training/testing/validation
  - Loss functions
  - Optimization
- Understand the principals of artificial neural network (ANN)
- Implement a simple ANN model, trained using gradient descent algorithm
  - Using Python Language
- Understand PyTorch basics

0

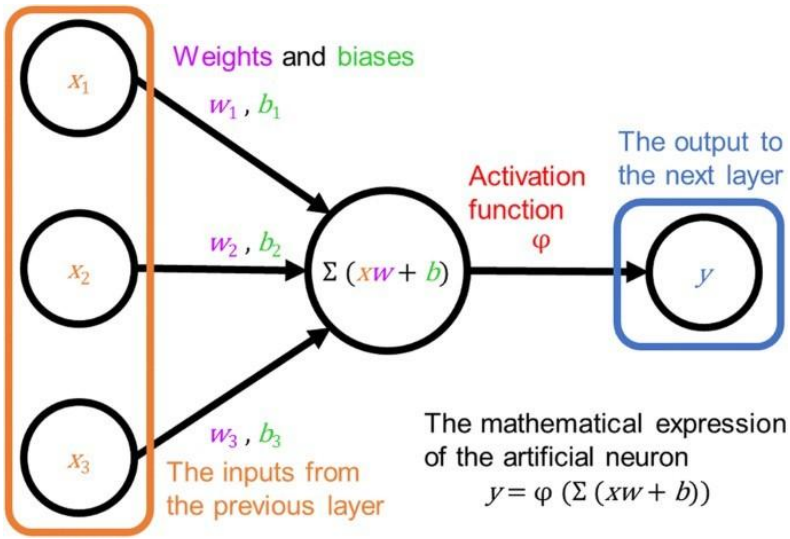
Deep Learning

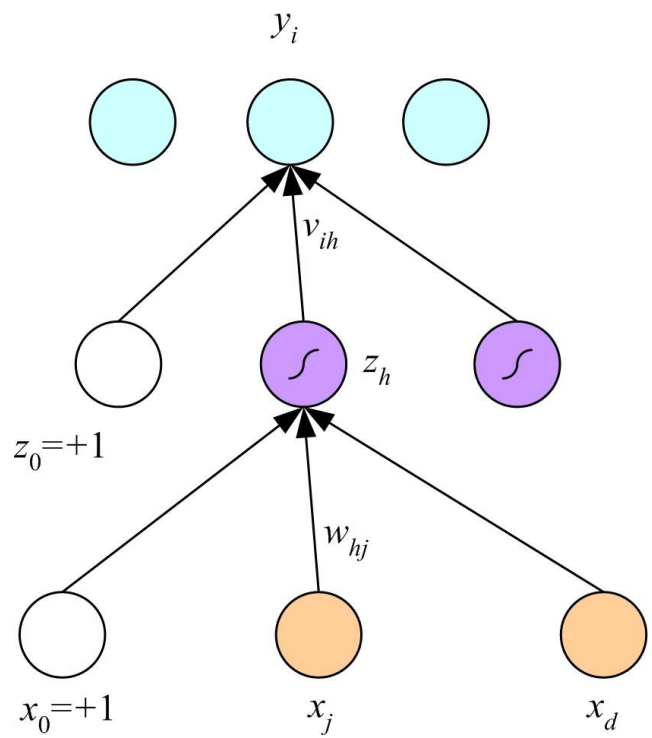
- Paradigm Shift
  - Feature engineering → feature learning



0

Deep Learning





Wataya, Tomohiro, et al. "Introduction to deep learning: minimum essence required to launch a research." *Japanese journal of radiology* (2020): 1-15.

0

Deep Learning Basics

1. Losses for tasks
2. Training/validation/testing
3. Optimization
   - Gradient descent
   - Problems
   - Optimizers
4. Computational Graph
5. Backpropagation

1

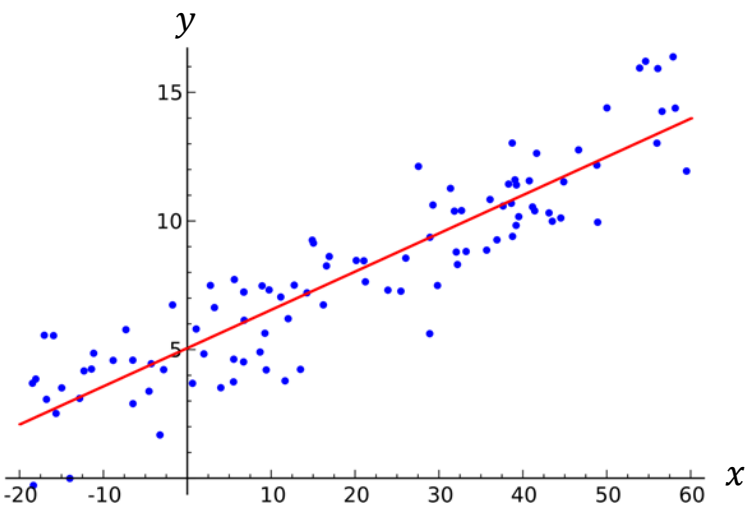
Performance Measure & DL

- Revisit the linear regression
  - Task: predict y from x
  - Experiences: blue dots
  - Univariate linear regression

    $h(x) = \theta_0 + \theta_1 x$
  - Parameters

    θ_0: **zero condition**

    θ_1: **gradient**



- **Machine learning**
  - Update parameters to achieve the best performance given data set

1

Performance Measure & DL

- **How to find optimal parameters**
  - Using performance measure

    Machine will find parameters minimizing (maximizing) the performance

    E.g. **$(h(x)-y)^2$ cost function**

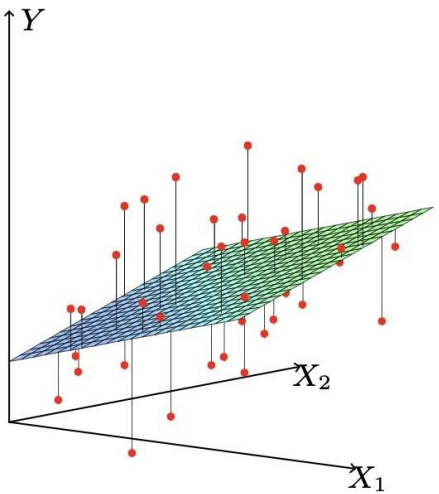
  - Ideas?

1

Performance Measure & DL

- Cost Function (Loss Function)
  - How good our model is → performance measure
  - Loss over dataset = average of loss over instances

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i,\theta), y_i)\,, \theta\text{: model parameters}$$



- Cost function for the regression problem
  - $(h(x)-y)^2$
  - $L = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i,\theta), y_i)\,, \theta\text{: model parameters}$

$$L_i(f(x_i,\theta), y_i) = (f(x_i,\theta) - y_i)^2$$

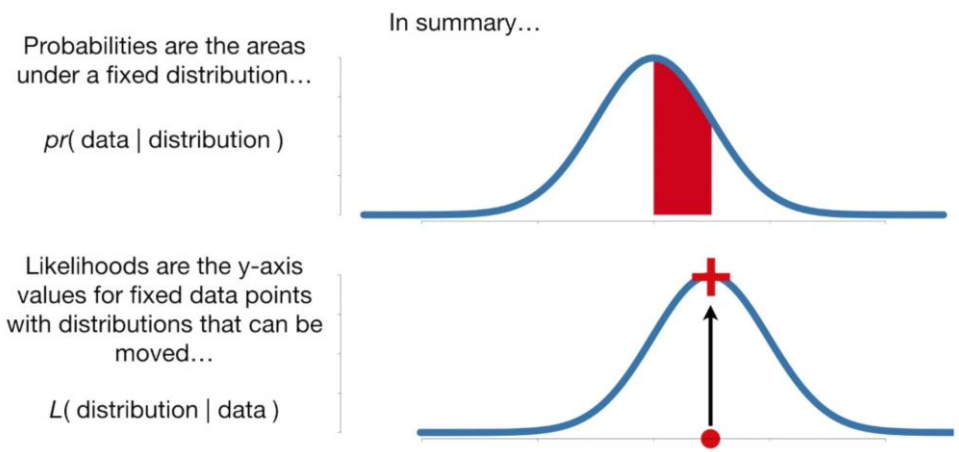
  - Mean squared error

Reference
http://cs231n.stanford.edu/
Elements of Statistical Learning

1

Performance Measure & DL

- Cost function for the classification problem
- $L = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, \theta), y_i)$, θ: $model\ parameters$
  - $f(x_i, \theta)$: p
  - Binary classification: **Bernoulli Distribution**
  - $h(Y = y, p) = p^y(1-p)^{1-y}$
  - Likelihood $= \log(\prod_{i=1}^{n} h(Y = y, p)) = \log(\prod_{i=1}^{n} p^y(1-p)^{1-y}) = \sum_{i=1}^{n} \log(p^y(1-p)^{1-y}) = \sum_{i=1}^{n} y\log(p)\,(1-y)\log(1-p)$
  -



1

Performance Measure & DL

- Regularization

Regularization is a set of techniques used to prevent overfitting.

It helps the model:

- Stay simple
- Avoid memorizing noise
- Generalize better to new data

1️⃣ L1 and L2 Regularization (Weight Penalty)

Instead of just minimizing loss:

$$Loss = Error$$

We modify it to:

$$Loss = Error + \lambda \times Penalty$$

Where:

- λ (lambda) controls how strong the penalty is
- The penalty depends on the size of weights

🔹 L2 Regularization

- Penalizes large weights
- Makes weights smaller
- Also called **weight decay**

🔹 L1 Regularization

- Pushes some weights to exactly zero
- Creates simpler models
- Helps with feature selection

👉 Idea: "Don't let weights become too large."

1

Performance Measure & DL

- Regularization
  - Prevent the model from overfitting on training data
  - <-> Data loss: to fit the model output to training data (label)
  - $L = \frac{1}{N} \sum_{i=1}^{N} L_i(f(x_i, \theta), y_i) + \lambda R(\theta)$, λ: $regularization\ strength$

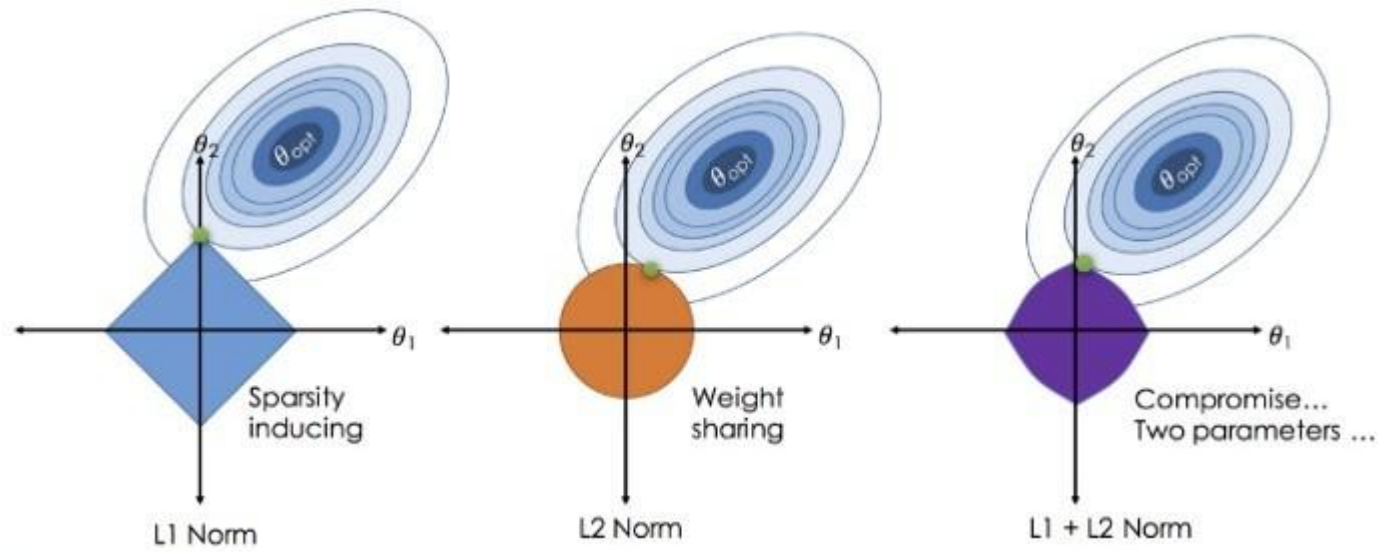
❖ Research about the in-depth and mathematical aspects of:

- L2 regularization
- L1 regularization
- Elastic net

1

Performance Measure & DL

- Regularization



Lasso | Ridge | Elastic Net

Zou, H., & Hastie, T. (2005). Elastic Net

1

Performance Measure & DL

- Revisit the loss function
  - **Performance measure**
  - $l_i = \left(y_i - f(x_i, \theta)\right)^2$
  - Expected error

$$E[y - \hat{y}^2] = \underbrace{E[\hat{y} - E[\hat{y}]]^2}_{\text{variance}} + \underbrace{\{E[\hat{y}] - E[y|x]\}^2}_{\text{bias}}, \hat{y} = f(x, \theta)$$

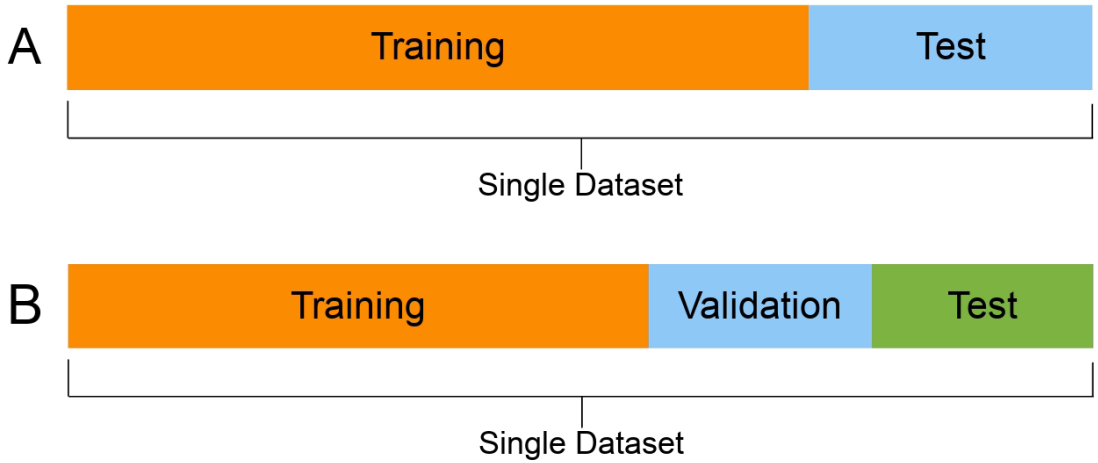
  - variance: how much $f(x)$ fluctuate around the expected value
  - bias: how much $f(x)$ is wrong disregarding the effect of varying sample

- Training = empirical risk minimization
  - "We can measure its performance on a known set of training data"

1

Performance Measure & DL

- Empirical risk minimization
  - Learning (knowledge) from dataset (experiences)
  - Generalizability: **model is applicable to new data?**
  - Model performance should be measured on independent data set (test set)



https://commons.wikimedia.org/wiki/File:ML_dataset_training_validation_test_sets.png

1

Performance Measure & DL

- Model Selection & Generalization
  - Learning is an **ill-posed problem**; **data is not sufficient to find a unique solution**

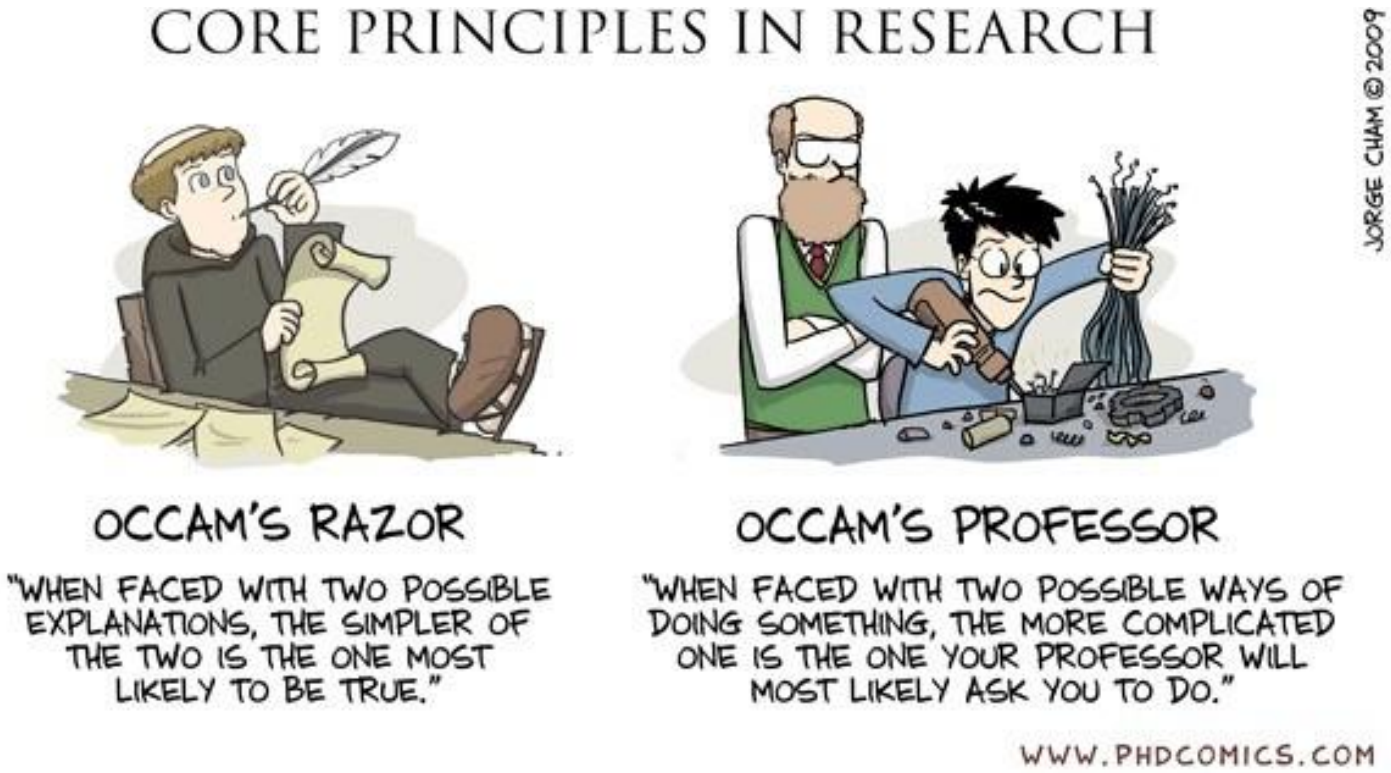
- There is a **trade-off between** three factors (Dietterich, 2003)
  - Complexity of H (hypothesis class), c (H)
  - Training set size, N
  - Generalization error, E, on new data

- Model performance improvement
  - **Decreasing the train error**
  - Increasing the number of training examples
  - Choose simple H ← Occam's Razor

1

Performance Measure & DL

- Occam's Razor



1

Performance Measure & DL

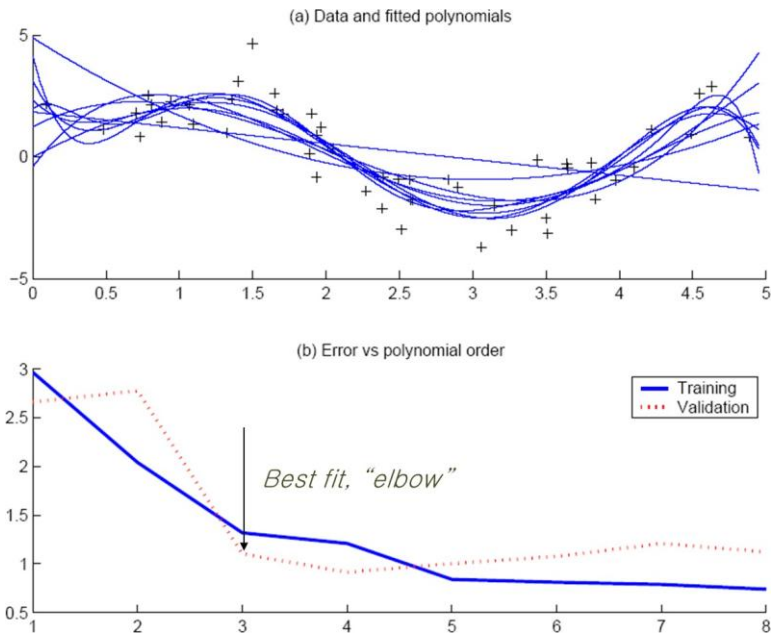
- Bias and Variance Dilemma
  - $f(x) = 2$: no variance and high bias
  - Model complexity: number of parameters, Vs dimensions
  - As we increase model complexity,
    bias decreases (a better fit to data) and
    variance increases (fit varies more with data)

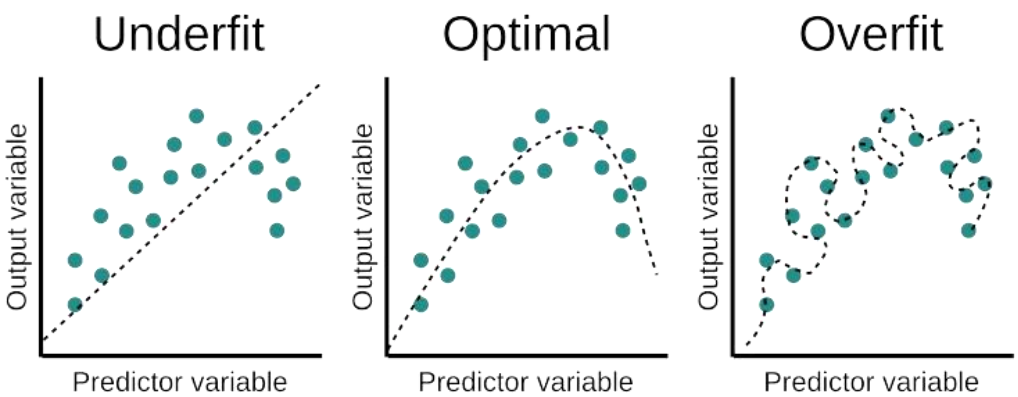
- Regularization
  - Reducing the variance

1

Performance Measure & DL

- Underfitting & overfitting
  - Underfitting : If a **too simple model is used (low complexity)**, fitting line does not approximate the distribution of points.
  - Overfitting : If a **overly complex model is chosen**, we can fit all training data perfectly. However, when a new dataset is added. It will not be able to make best predictions.

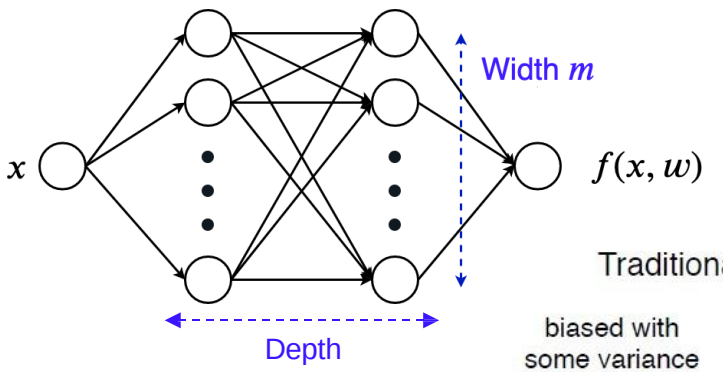


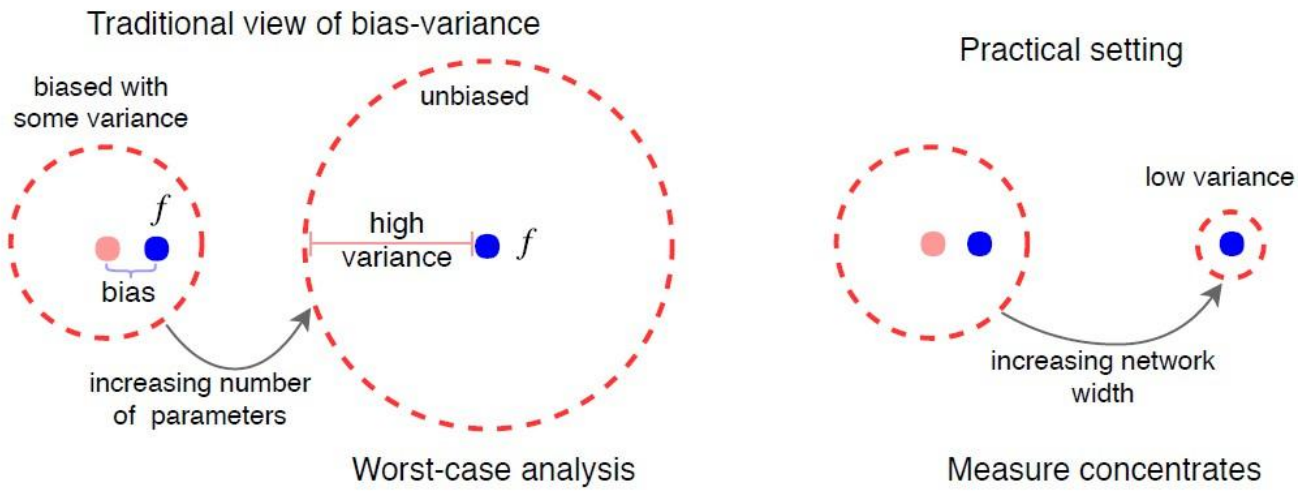


1

Performance Measure & DL

- Neal, Brady, et al. "A modern take on the bias-variance tradeoff in neural networks." *arXiv preprint arXiv:1810.08591* (2018).
  - **Network width = decrease bias and variance**
  - **Network depth = decrease bias, but variance increases**





1

Break

2

Loss Function

- Performance measure
  - Depends on machine tasks
  - Classification, regression, segmentation, …
  - Problem definition

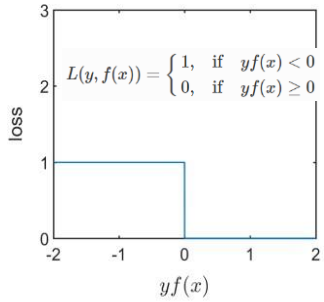
Category	Task/Application	Loss function
Traditional machine learning	Classification problem	0–1 loss, Perceptron loss, Logarithmic loss, Exponential loss, Sigmoid cross entropy loss, Softmax cross entropy loss, Hinge loss, Ramp loss, Pinball loss, Truncated pinball loss, Rescaled hinge loss
	Regression problem	Square loss, Absolute loss, Huber loss, Log-cosh loss, Quantile loss, ϵ-insensitive loss
	Unsupervised learning	Square error, Distance error, Reconstruction error, Negative variance
Deep learning	Object detection	IoU loss, Generalized IoU loss (GIoU loss), Smooth L1 loss, Focal loss
	Face recognition	Contrastive loss, Triplet loss, Center loss, Angular softmax loss (A-Softmax loss), Additive margin softmax loss (AM-Softmax loss), Additive angular margin loss (ArcFace loss)

Wang, Qi, et al. "A comprehensive survey of loss functions in machine learning." *Annals of Data Science* (2020): 1-26.

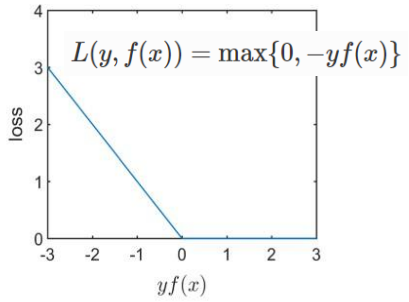
2

Loss Function

Classification Loss

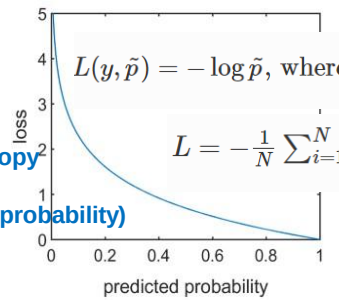


(a) 0-1 loss

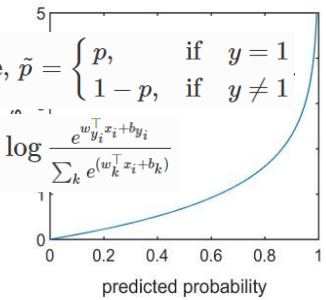


(b) perceptron loss

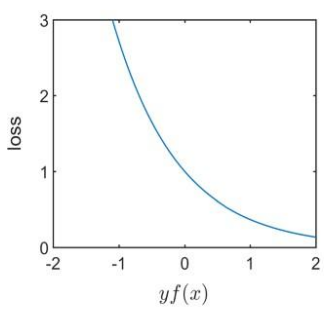
Cross-entropy Loss
(Maximize probability)



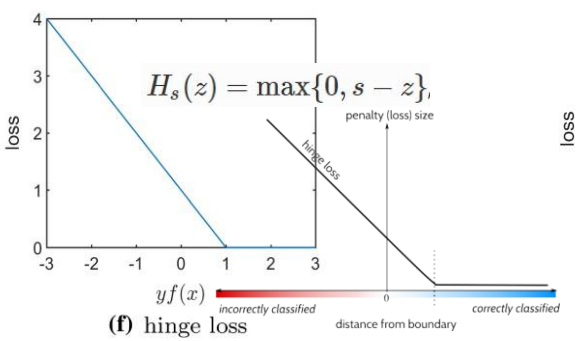
(c) logarithmic loss(label=1)



(d) logarithmic loss(label=-1)

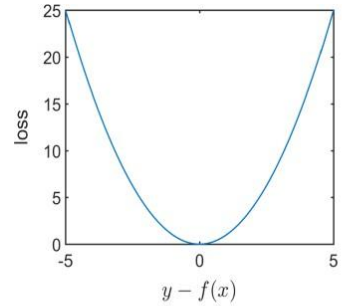


(e) exponential loss

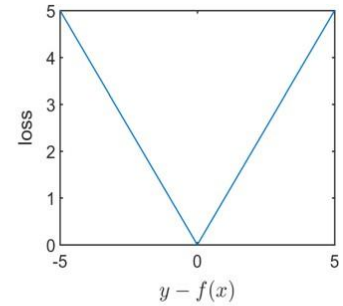


(f) hinge loss

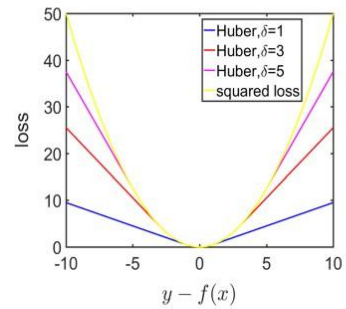
Regression Loss



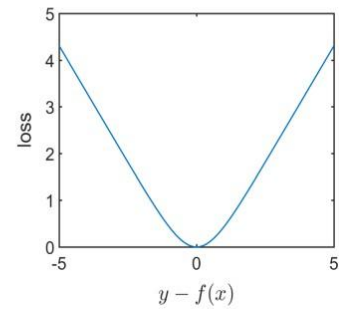
(a) square loss



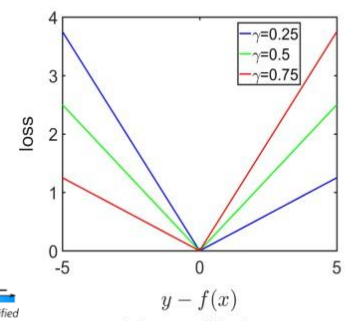
(b) absolute loss



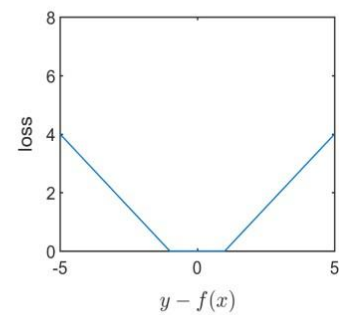
(c) Huber loss



(d) log-cosh loss



(e) quantile loss

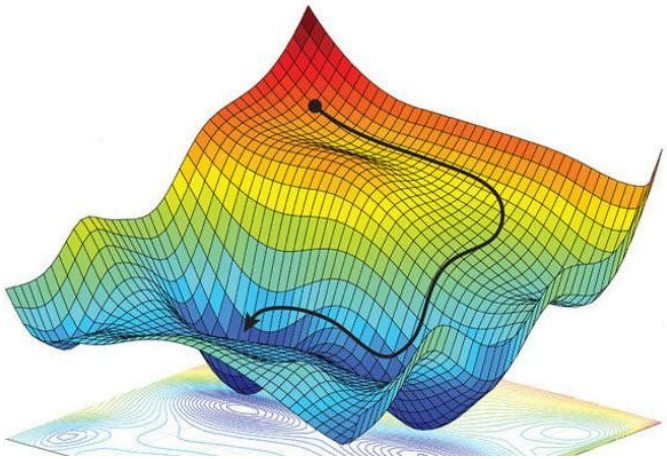


(f) $\epsilon-$insensitive loss($\epsilon = 1$)

3

Optimization

- How to find optimal parameters
  - Minimizing loss (cost)
  - Maximizing probability



- Gradient descent algorithm
  - more parameters → larger space
  - Gradient: vector of partial derivatives along each dimension
  - $h(x) = \theta_0 + \theta_1 x$

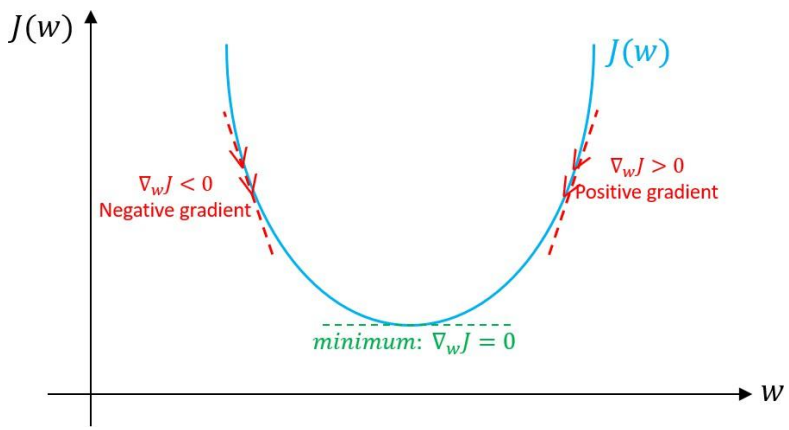
$$\boldsymbol{\theta_j = \theta_j - \eta \frac{\partial L}{\partial \theta_j}}$$

3

Optimization

- Gradient descent algorithm
  1. Initialize trainable parameters (weight, bias)

     Random function
  2. Calculate the cost and its gradient at given parameters
  3. Update parameters with the gradient and learning rate



$$\boldsymbol{\theta_j = \theta_j - \eta \frac{\partial L}{\partial \theta_j}}$$

η: learning rate

https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3

3

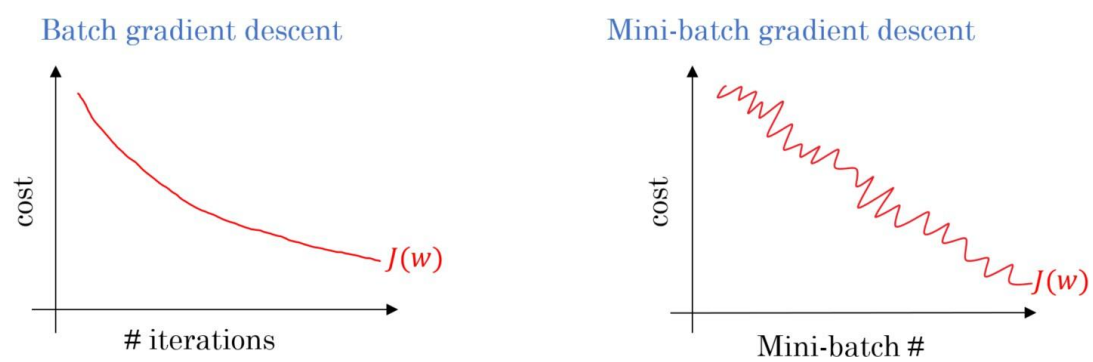
Optimization

- **Stochastic Gradient Descent**
  - True gradient of the cost function is approximated by a gradient at a single instance
  - "On-line gradient descent"
- Pros
  - Frequent (parameter) updates → faster learning sometimes
  - Simple to implement
  - Noisy update can help avoid local minima → generalization
- Cons
  - Frequent update of the model → computationally expensive
  - Higher variance over epochs by noisy gradient signal
  - Difficult to settle ← noisy gradient

3

Optimization

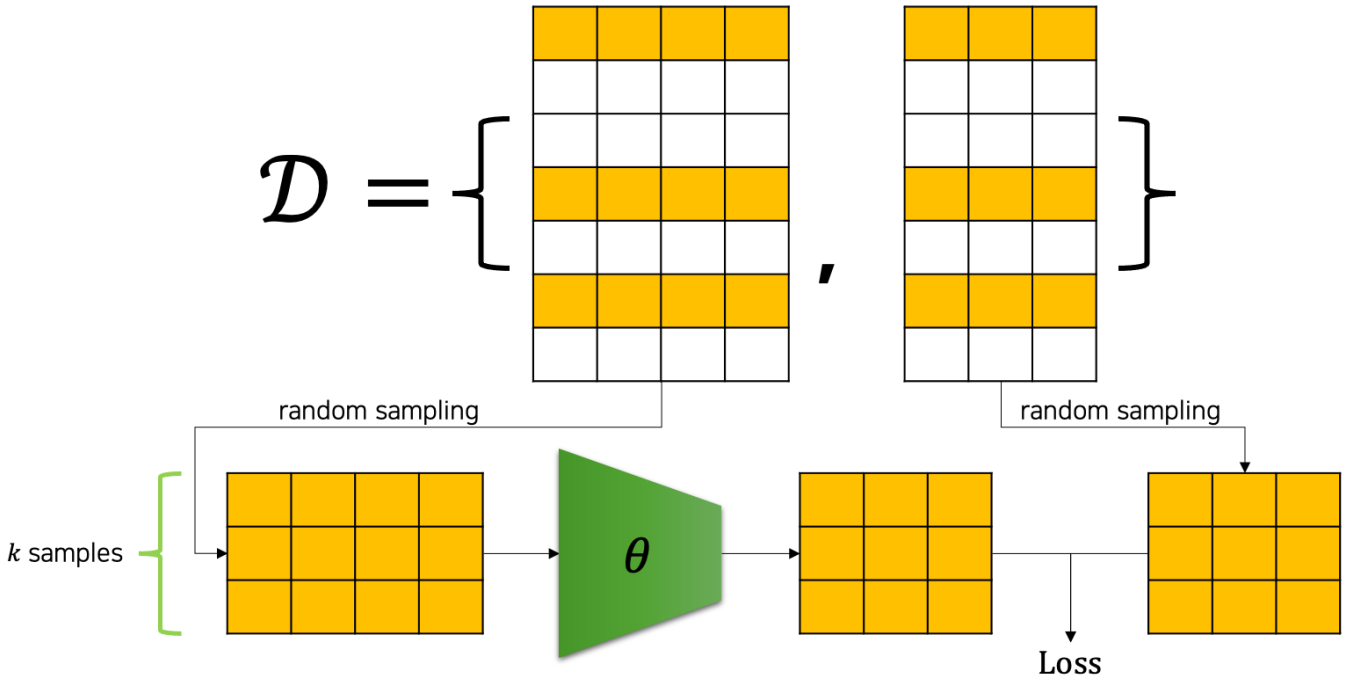
- **Batch Gradient Descent**
  - Compute gradient over all training samples at each step
  - Fewer updates → computationally efficient, stable error gradient
  - Parallel processing for error calculation and parameter update
- Cons
  - Stable error gradient → premature convergence
  - Large memory is required for the entire training dataset
  - Slow training
- If m is too large, we can do the same thing with smaller set of samples → **mini-batch**



3

Optimization

- Mini-batch



https://kh-kim.github.io/nlp_with_deep_learning_blog/docs/1-10-sgd/01-sgd

3

Optimization

- Gradient computation
  - Numerical gradient

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{f(x+h) - f(x-h)}{2h}$$

Approximation, slow

  - Analytic gradient

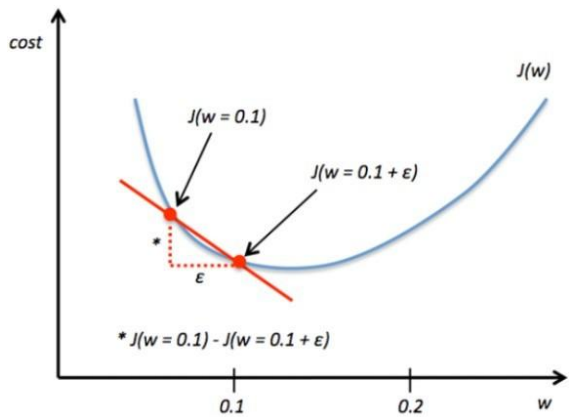
$$f(x_0, x_1) = x_0^2 + x_1^2$$

$$\frac{\partial f}{\partial x_0} = 2x_0$$

$$\frac{\partial f}{\partial x_1} = 2x_1$$

Exact, fast

  - Gradient check: analytic gradient = numerical gradient?



3

Optimization

- Numerical differentiation
  - Partial derivatives for multiple parameters

- Hands-on
  - $f(x) = 0.1x^2 + x$
  - Implement $f(x)$
  - Draw the function using matplotlib

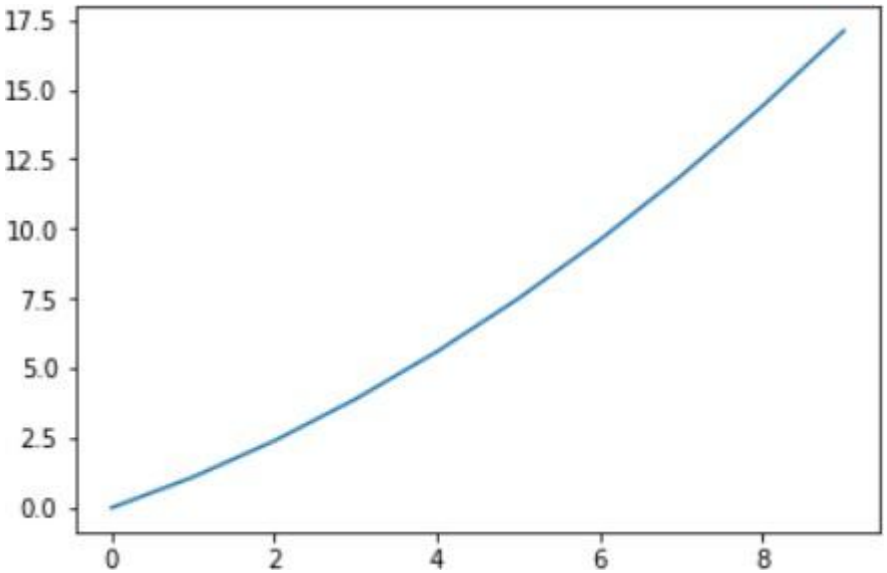
```
import matplotlib.pyplot as plt
fig = plt.figure(frameon=False)
plt.plot(x, y)
plt.show()
```

  - Calculate the derivative of $f(x)$, when $x = 3$

3

Optimization

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return 0.1*(x**2) + x

x = np.arange(0.0, 10.0, 1)
y = f(x)

fig = plt.figure(frameon=False)
plt.plot(x, y)
plt.show()
```




```
def numerical_diff(f, x):
    h = 10e-4
    return (f(x+h)-f(x))/h

print(numerical_diff(f, x=3))
```

3

Optimization

- Hands-on
  - $f(x_0, x_1) = 2x_0^2 + x_0 x_1^2$
  - Implement $f(x)$
  - Calculate the partial derivative of $f(x_0, x_1)$ with respect to x_0, when $x_0 = 3$ and $x_1 = 4$

    using numerical differentiation

    using $\frac{\partial f(x_0, x_1)}{\partial x_0} = 4x_0 + x_1^2$

3

Optimization

```
def f2(x0, x1):
    return 2*(x0**2) + x1**2

print(f2(x0=3, x1=4))
```

Numerical differentiation

```
def f2_x0(x0):
    x1 = 4
    return 2*(x0**2) + x0*(x1**2)

print(numerical_diff(f2_x0, 3))
```

Partial derivative with respect to $x_{\$}$

```
def df2_x0(x0):
    x1 = 4
    return 4*x0 + x1**2

print(df2_x0(3))
```

3

Optimization

- Hands-on: gradient descent algorithm
- $f(x_0, x_1) = x_0^2 + x_1^2$
  - $x_0 = x_0 - \eta \frac{\partial f}{\partial x_0}$
  - $x_1 = x_1 - \eta \frac{\partial f}{\partial x_1}$
  - η: learning rate
  - using numerical gradient: $[\frac{\partial f}{\partial x_0}, \frac{\partial f}{\partial x_1}]$
- Find the minima of $f(x_0, x_1)$
  - Initial values: $x_0 = 3, x_1 = 3$

    x = np.array([3.0, 3.0])
  - $\eta = 0.1$
  - step_num = 100

```python
import numpy as np

def f(x):
    return x[0]**2 + x[1]**2

def gradient(f, x):
    h = 10e-4
    grad = np.zeros_like(x)

    for i in range(x.size):
        tmp = x[i]
        x[i] = tmp + h
        fx1 = f(x)
        x[i] = tmp - h
        fx2 = f(x)
        grad[i] = (fx1 - fx2)/(2.0*h)
        x[i] = tmp

    return grad

def gradient_descent(f, init_x, lr, step_num):
    x = init_x
    # fill space
```

```python
    return x
```

3

Optimization

```python
import numpy as np

def f(x):
    return x[0]**2 + x[1]**2

def gradient(f, x):
    h = 10e-4
    grad = np.zeros_like(x)

    for i in range(x.size):
        tmp = x[i]
        x[i] = tmp + h
        fx1 = f(x)
        x[i] = tmp - h
        fx2 = f(x)
        grad[i] = (fx1 - fx2)/(2.0*h)
        x[i] = tmp

    return grad

def gradient_descent(f, init_x, lr, step_num):
    x = init_x
    # fill space




    return x
```

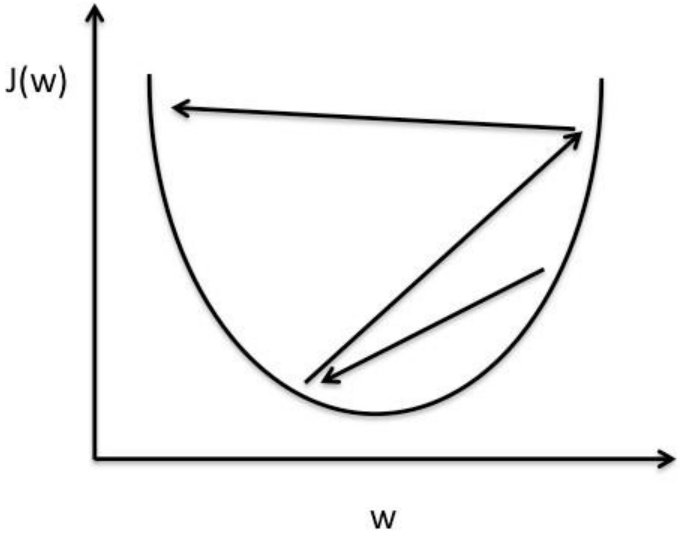
```python
# Test code
init_x = np.array([3.0, 3.0])
lr = 0.1
step_num = 100
x = gradient_descent(f, init_x, lr, step_num)
print(x)
```

- Result?
- What if $\eta = 100$
- What if $\eta = 0.001$
- **Learning rate**
  - **Hyper-parameter** that controls how much the parameters are adjusted with respect to the error gradient

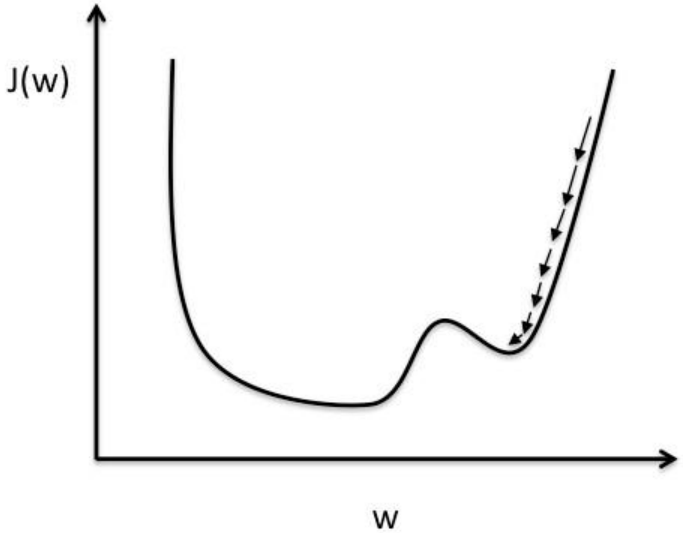
3

Optimization

- In gradient descent algorithm
  - Risk of local minimum



Large learning rate: Overshooting.

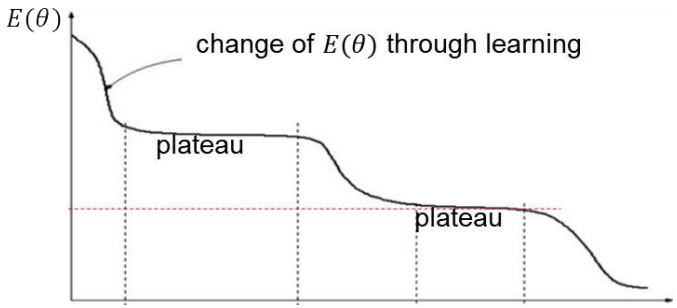


Small learning rate: Many iterations until convergence and trapping in local minima.

3

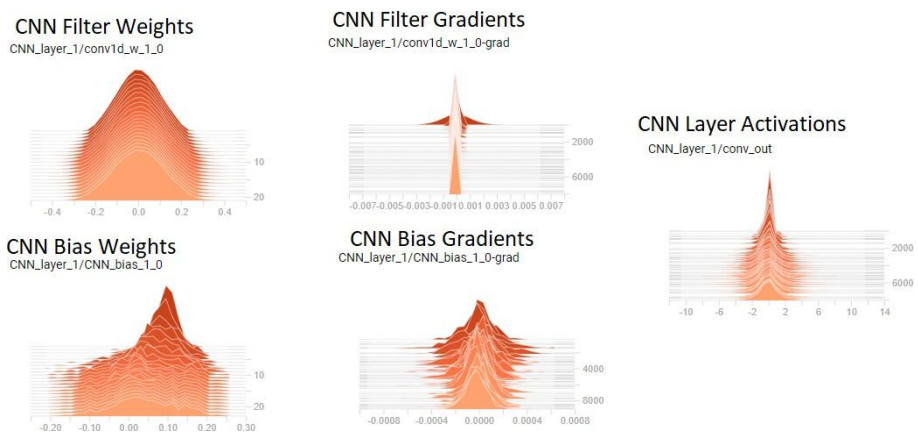
Optimization

- Learning curve of gradient descent method
  - Plateau problem



  - Vanishing gradient problem

    Magnitude of gradient decreases too much



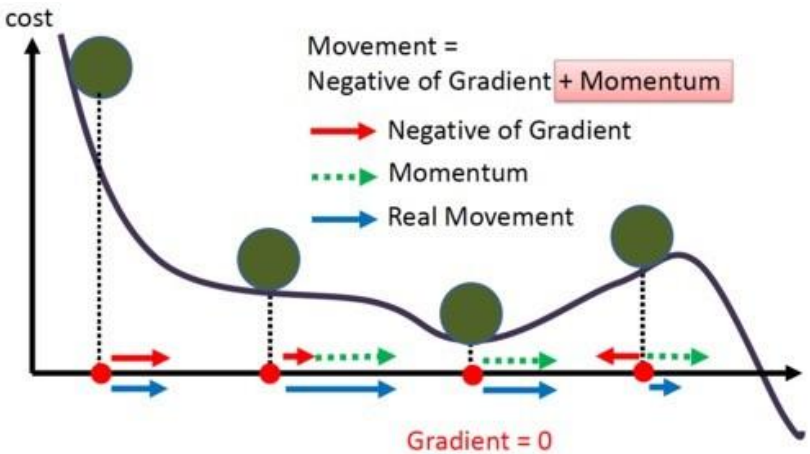
3

Optimization

- Improving convergence
  - Avoid local minima

- **Momentum**

$$\Delta w_i^t = -\eta \frac{\partial E^t}{\partial w_i} + \alpha \Delta w_i^{t-1}$$



$$\nu_1 = -G_1$$

$$\nu_2 = -0.9 * G_1 - G_2$$

$$\nu_3 = -0.9 * (0.9 * G_1 - G_2) - G3 = -0.81 * (G_1) - (0.9) * G_2 - G_3$$

https://medium.com/analytics-vidhya/momentum-rmsprop-and-adam-optimizer-5769721b4b19

3

Optimization

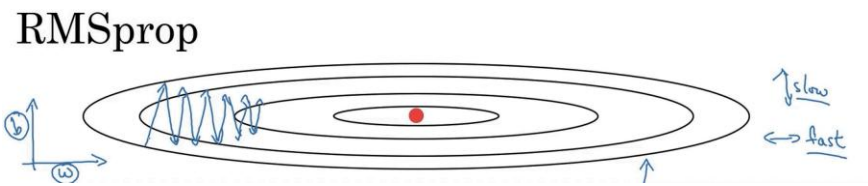
- **RMSProp**
  - Root Mean Square Propagation by Geoffrey Hinton
  - Keeping **the moving average of the squared gradients** for each parameter and **dividing the gradient by square root the mean square**

    The square root = the magnitudes of recent gradients
  - No need to adjust learning rate

$$\nu_t = \rho\nu_{t-1} + (1-\rho) * g_t^2$$

$$\Delta\omega_t = -\frac{\eta}{\sqrt{\nu_t + \epsilon}} * g_t$$

$$\omega_{t+1} = \omega_t + \Delta\omega_t$$



  - **Accelerating the movement along one dimension with low gradient** and **slowing down the movement along other dimension**
  - Avoid convergence to local minimum

3

Optimization

- **Adam**
  - Combines Momentum and RMSProp

    Momentum: update the parameters towards the direction of the velocity of gradient

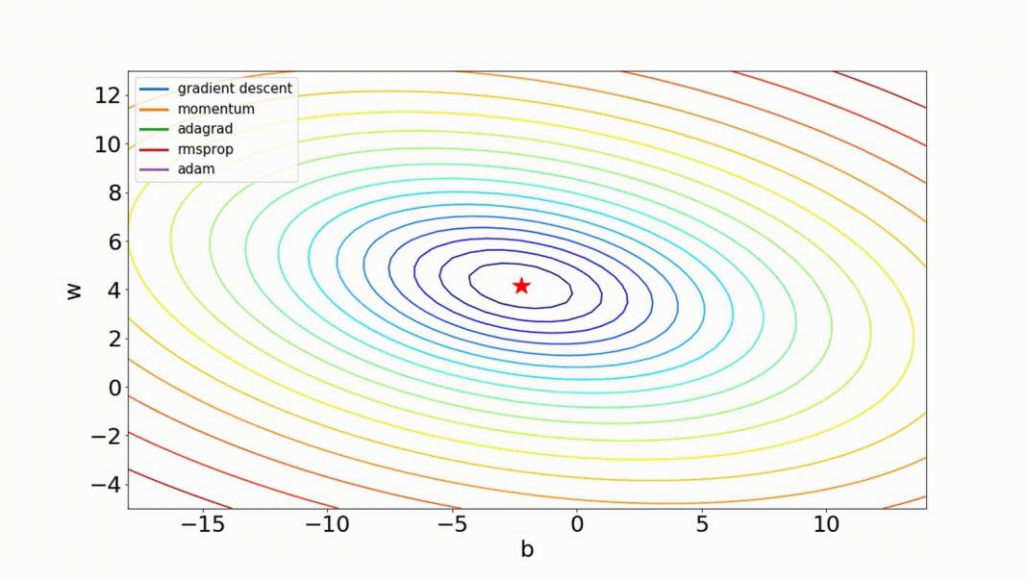
    RMSProp: scale the current direction by the sum of the moving average of gradients

$$\nu_t = \beta_1 * \nu_{t-1} - (1 - \beta_1) * g_t$$

$$s_t = \beta_2 * s_{t-1} - (1 - \beta_2) * g_t^2$$

$$\Delta\omega_t = -\eta \frac{\nu_t}{\sqrt{s_t + \epsilon}} * g_t$$

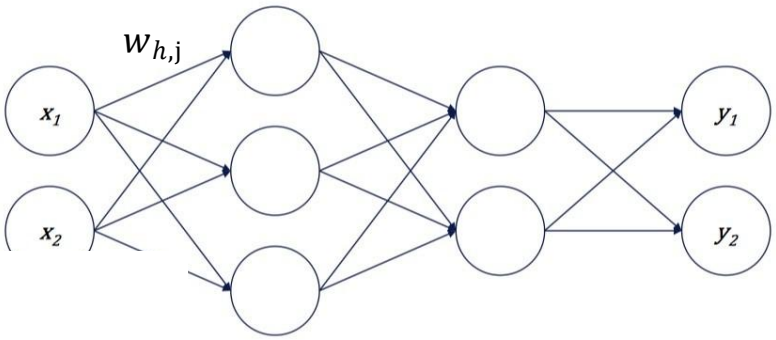
$$\omega_{t+1} = \omega_t + \Delta\omega_t$$



3

Gradient Computation for DL

- Gradient
  - $[\frac{\partial L}{\partial w_{h,j}}]$ to learn parameters



$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

3

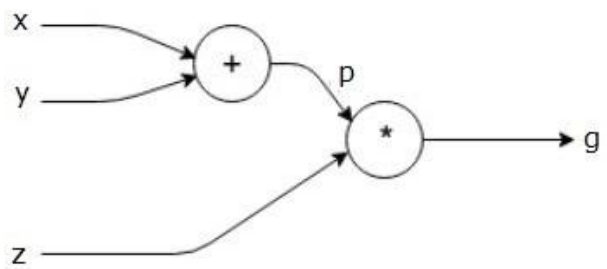
Break

4

Computational Graph

- DL implementation
  - Computational graph and backpropagation
  - Update the parameters

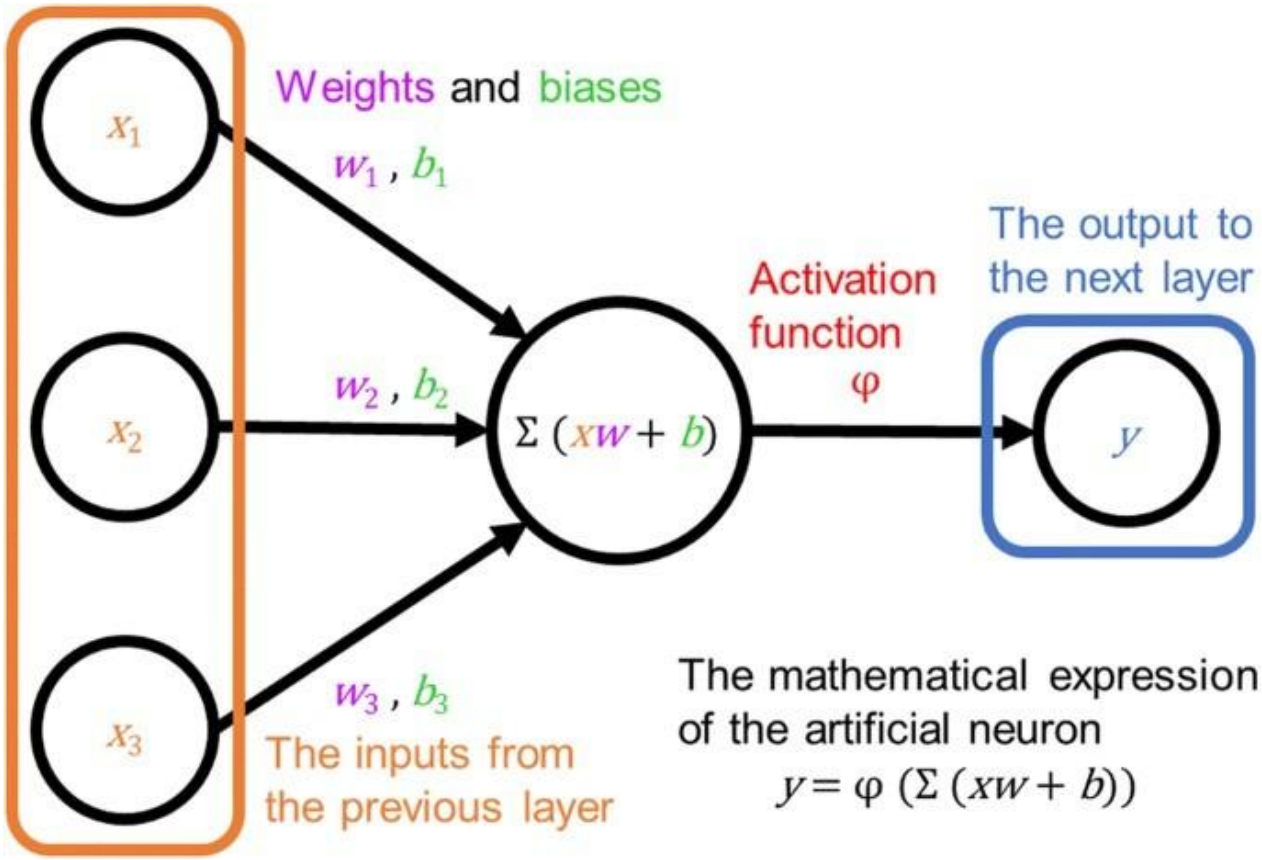
- Computational Graph
  - **Descriptive language of deep learning models**
  - To do two types of operation: forward and backward
  - Graph: a set of nodes and edges



- Components
  - Node: a {tensor, matrix, vector, scalar} value
  - Edge: a function argument (data dependency)
  - Node with an incoming edge: a function of edge's tail node

4

Computational Graph



4

Computational Graph

- **Chain rule**
  - "a formula that expresses the derivative of the composition of two differentiable functions, f and g"
  - $h = f \circ g$: function composition
  - $h^3 = f'(g)g'$: derivative of h
  - $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x}$

$$\frac{d}{dx}\left[\left(f(x)\right)^n\right] = n\left(f(x)\right)^{n-1} \cdot f'(x)$$

$$\frac{d}{dx}\left[f\left(g(x)\right)\right] = f'\left(g(x)\right)g'(x)$$

Calcworkshop.com

- Neural network
  - Chain of (non-linear) functions
  - Update each parameter using chain rule

4

Computational Graph

- Hands-on
  - $z = t^2$, $t = x + y$, then $\frac{\partial z}{\partial x}$?
  - Draw a computational graph

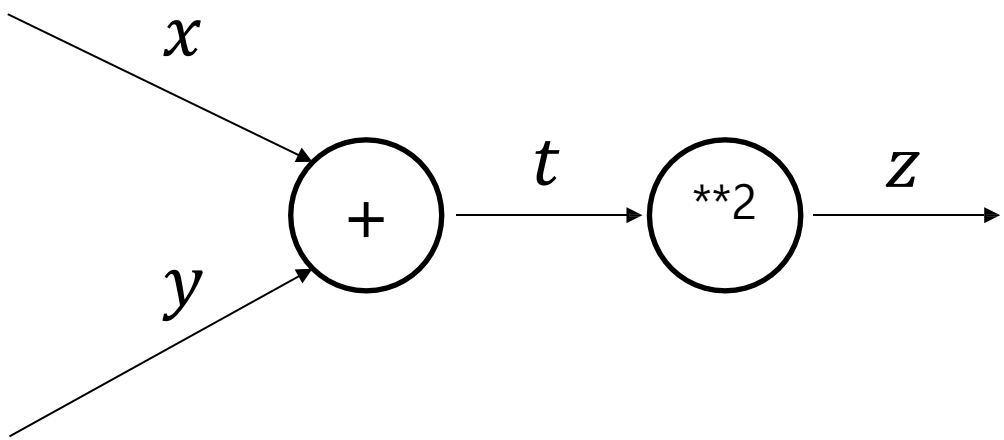
4

Computational Graph

- Hands-on
  - $z = t^2$, $t = x + y$, then $\frac{\partial z}{\partial x}$?

    $\frac{\partial z}{\partial t} = 2t$, $\frac{\partial t}{\partial x} = 1$

    $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial t}\frac{\partial t}{\partial x} = 2\mathrm{t} = 2(\mathrm{x} + \mathrm{y})$
  - Draw a computational graph



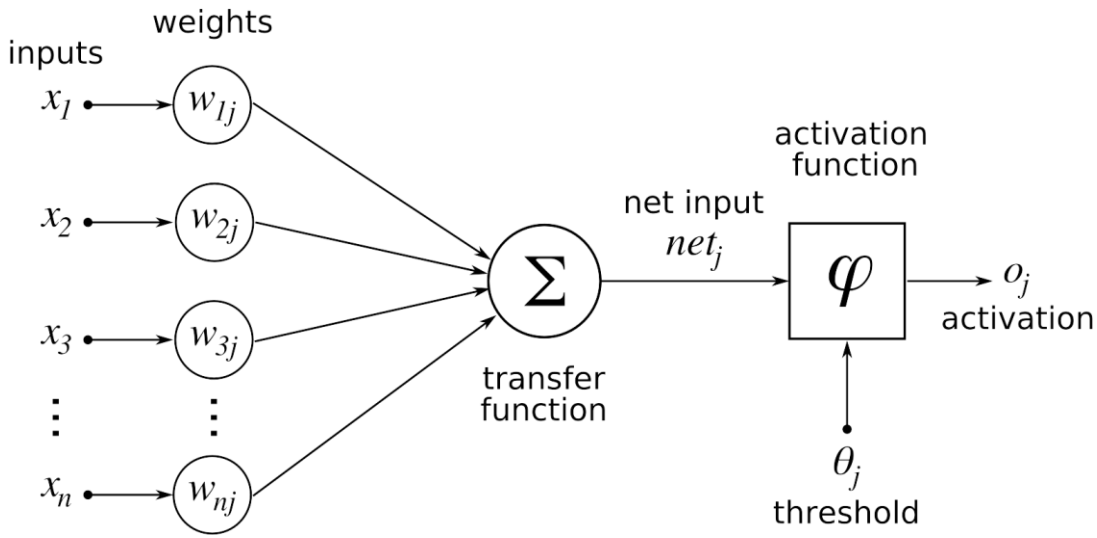
4

Computational Graph

- Backpropagation
  - "Computes the gradient of the loss function with respect to the weights of the network for a single input–output"
  - Gradient descent algorithms for multi-layer networks

Update parameters to minimize loss

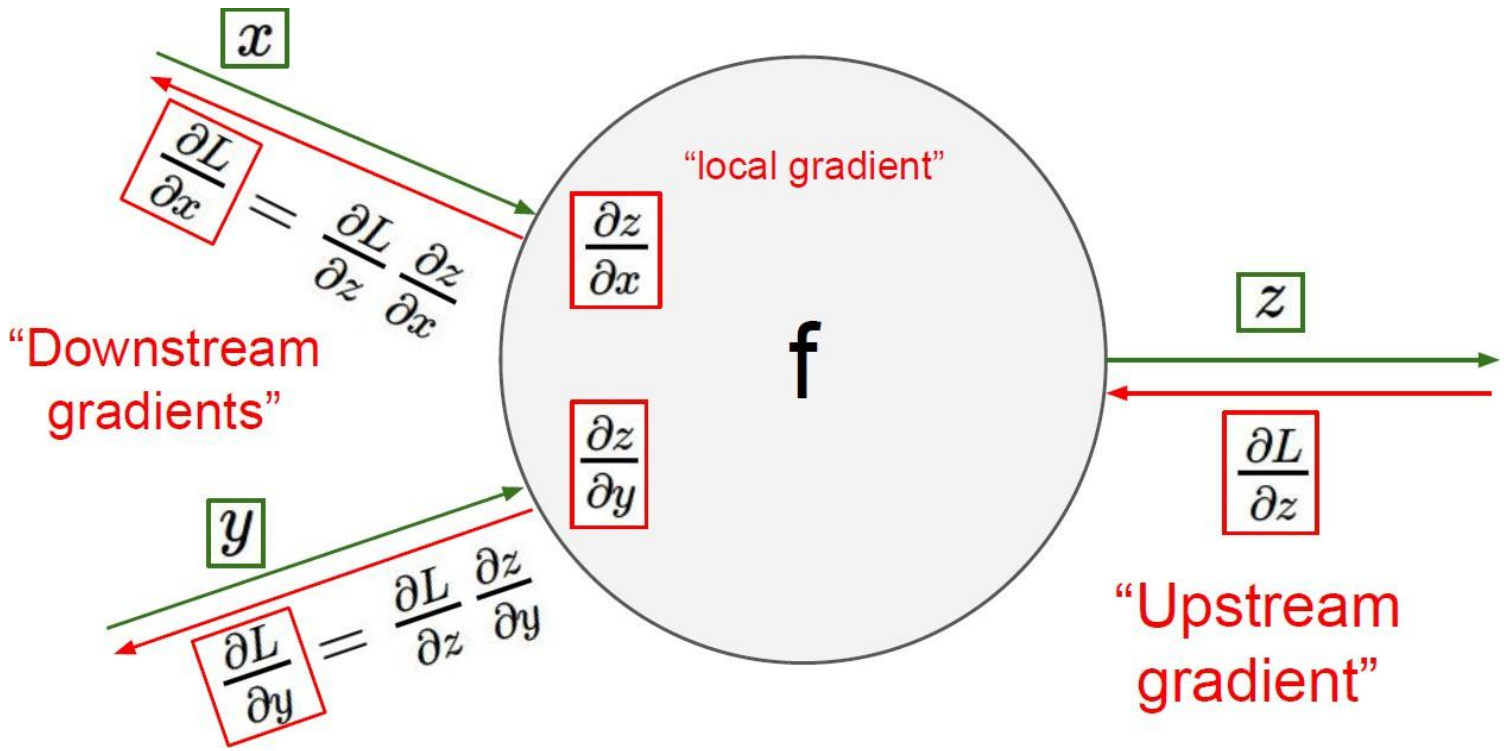
$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j}\frac{\partial o_j}{\partial \mathrm{net}_j}\frac{\partial \mathrm{net}_j}{\partial w_{ij}}$$



https://en.wikipedia.org/wiki/Backpropagation

4

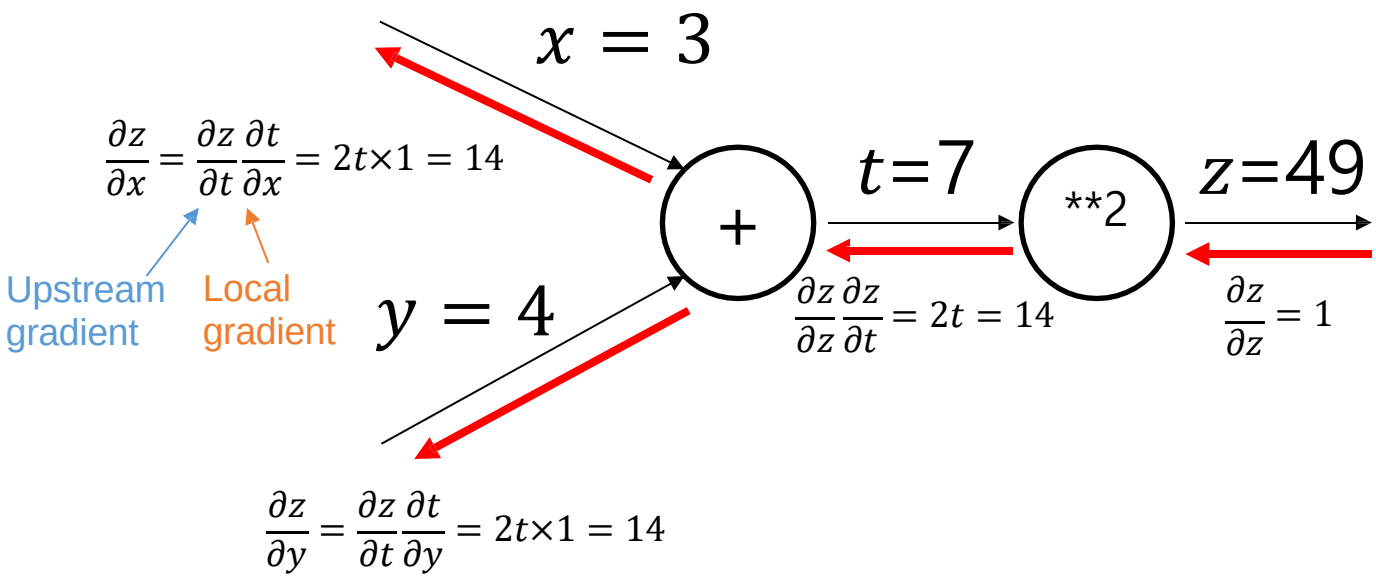
Backpropagation



4

Backpropagation

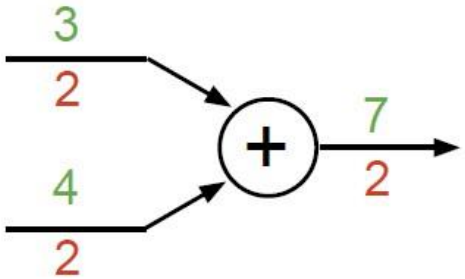
- Backpropagation
  - $z = t^2, t = x + y$
  - To update x and y
  - $\frac{\partial z}{\partial x}, \frac{\partial z}{\partial y}, \frac{\partial z}{\partial t}$



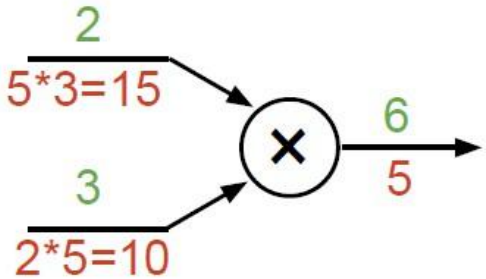
Backpropagation

- Patterns in gradient flow

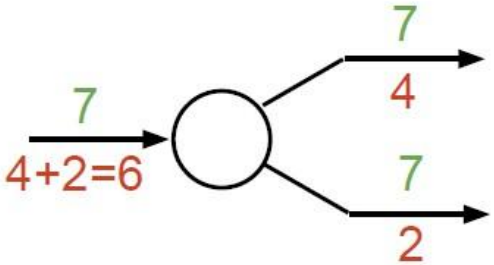
add gate: gradient distributor



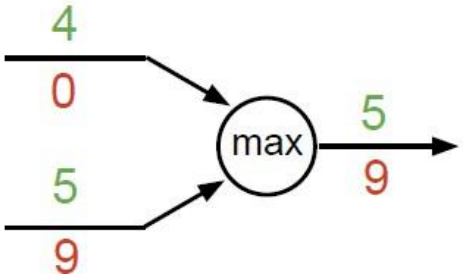
mul gate: "swap multiplier"



copy gate: gradient adder



max gate: gradient router



4

Backpropagation

- Hands-on
  - Implementation: Add gate and Mul gate

```python
class AddGate:
    def __init__(self) -> None:
        pass

    def forward(self, x, y):
        return x + y

    def backward(self, dupstream):
        # fill space
```

```python
class MulGate:
    def __init__(self) -> None:
        self.x = None
        self.y = None

    def forward(self, x, y):
        self.x = x
        self.y = y
        return x*y

    def backward(self, dupstream):
        # fill space
```

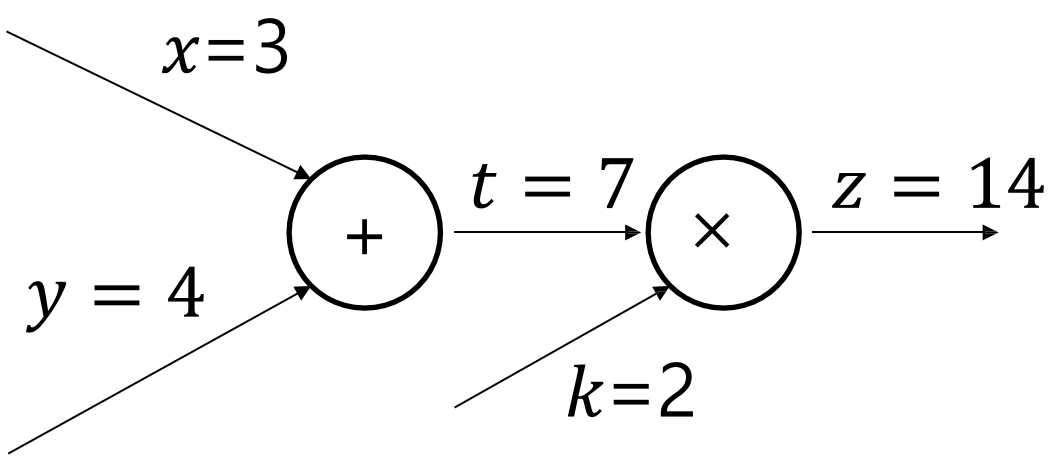
```python
        return dx, dy

x = 3
y = 4

add_op = AddGate()
print(add_op.forward(x, y))
dout = 1
print(add_op.backward(dout))
```

```python
        return dx, dy
```

4

Backpropagation

- Hands-on
  - Implementation: Add gate and Mul gate

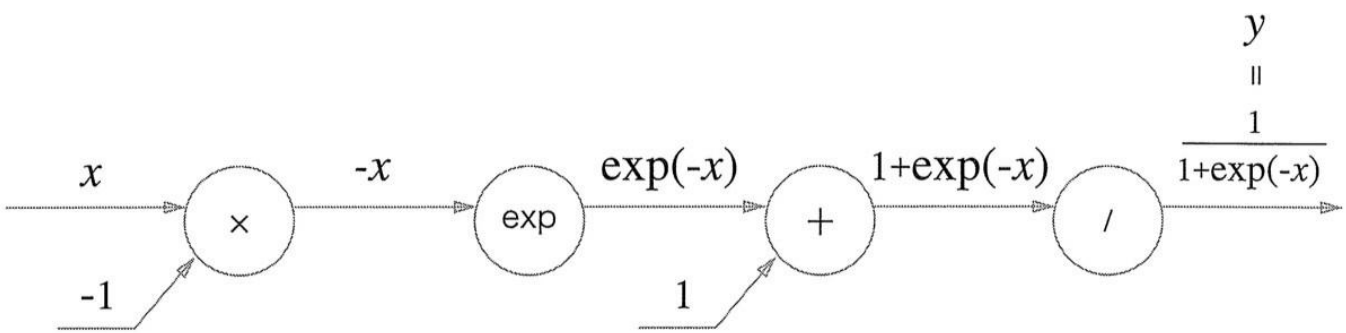



```python
x = 3
y = 4
k = 2
add_op = AddGate()
mul_op = MulGate()

# build a chain
t = add_op.forward(x, y)
z = mul_op.forward(t, k)
print(x, y, t, z)

# backpropagation
dz = 1
dt, dk = mul_op.backward(dz)
dx, dy = add_op.backward(dt)

print(dz, dt, dx, dy)
```

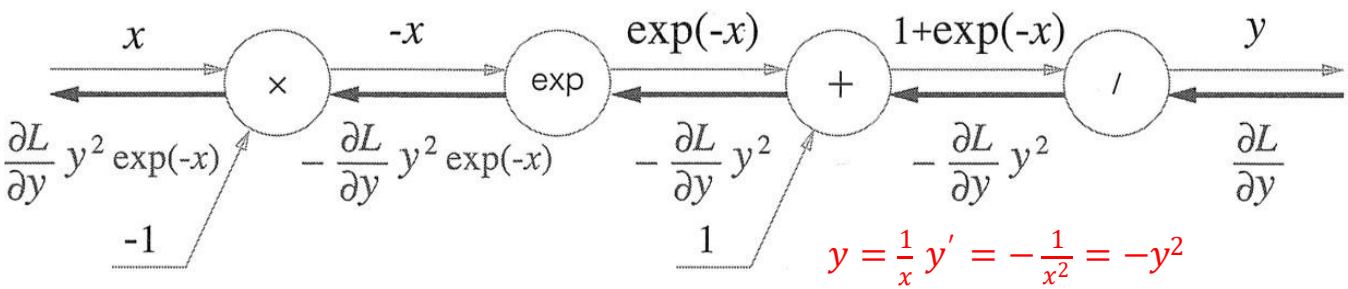
4

Backpropagation

- Sigmoid activation function
  - $y = \frac{1}{1+e^{\&x}}$
  - Computational graph



  - Backpropagation

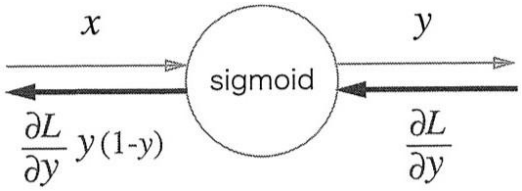


4

Backpropagation

- Sigmoid activation function
  - $\frac{\partial L}{\partial y} y^2 e^{-x} = \frac{\partial L}{\partial y} (\frac{1}{1+e^{\&x}})^2 e^{-x} = \frac{\partial L}{\partial y} \frac{1}{1+e^{\&x}} \frac{e^{\&x}}{1+e^{\&x}}$

$$= \frac{\partial L}{\partial y} y(1-y)$$



  - Using only the output of the forward propagation

```python
import numpy as np

class Sigmoid:
    def __init__(self) -> None:
        self.fout = None

    def forward(self, x):
        self.fout = 1.0/(1.0 + np.exp(-x))
        return self.fout

    def backward(self, dout):
        dx = dout * self.fout * (1.0 - self.fout)
        return dx
```

```python
x = 3
y = 4
k = 2
add_op = AddGate()
mul_op = MulGate()
sigmoid_op = Sigmoid()

# build a chain
t = add_op.forward(x, y)
z = mul_op.forward(t, k)
o = sigmoid_op.forward(z)
print(x, y, t, z, o)

# backpropagation
do = 1
dz = sigmoid_op.backward(do)
dt, dk = mul_op.backward(dz)
dx, dy = add_op.backward(dt)

print(dz, dt, dx, dy)
```

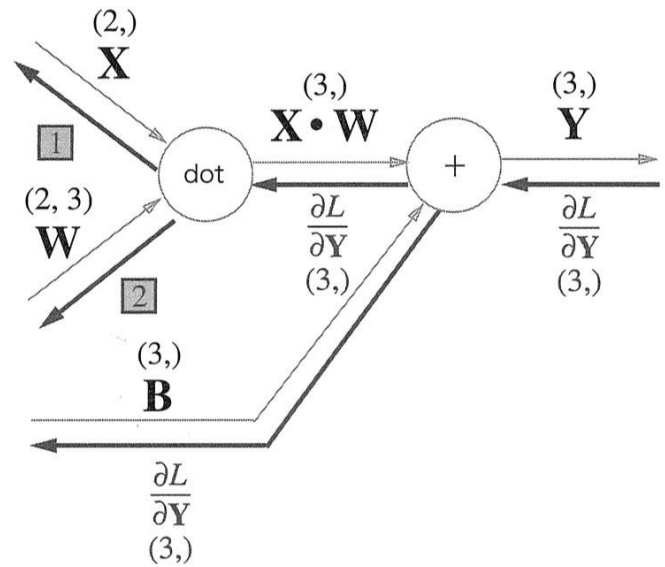
4

Backpropagation

- Affine layer
  - Dot product
  - Affine transformation
  - Weights × features
  - Backpropagation

$$\boxed{1}\quad \underset{(2,)}{\frac{\partial L}{\partial \mathbf{X}}} = \underset{(3,)}{\frac{\partial L}{\partial \mathbf{Y}}}\ \underset{(3,\ 2)}{\mathbf{W}^{\mathrm{T}}}$$

$$\boxed{2}\quad \underset{(2,\ 3)}{\frac{\partial L}{\partial \mathbf{W}}} = \underset{(2,\ 1)}{\mathbf{X}^{\mathrm{T}}}\ \underset{(1,\ 3)}{\frac{\partial L}{\partial \mathbf{Y}}}$$



$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \cdot \mathbf{W}^T \qquad W = \begin{pmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \end{pmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^T \cdot \frac{\partial L}{\partial \mathbf{Y}} \qquad W^T = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{pmatrix}$$

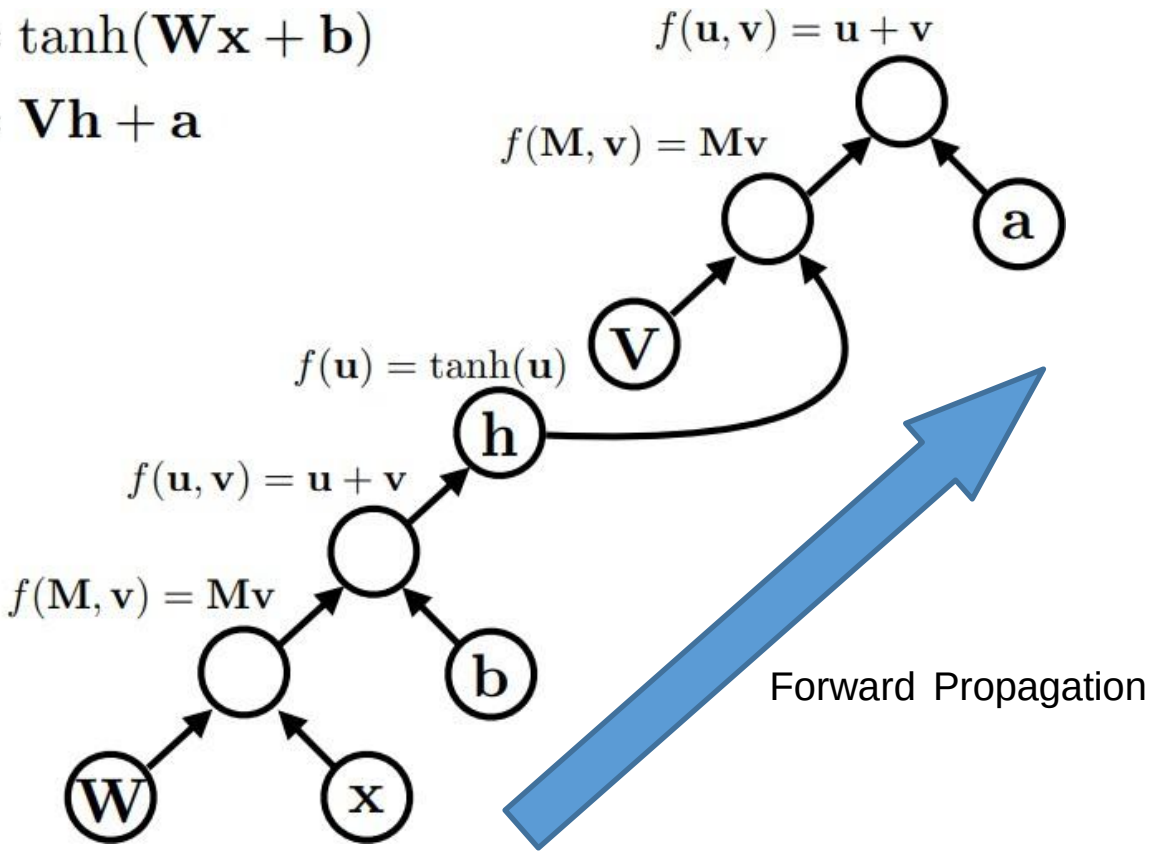
4

Backpropagation

- Computational Graph of Multi-layer perceptron

$$\mathbf{h} = \tanh(\mathbf{W}\mathbf{x} + \mathbf{b})$$

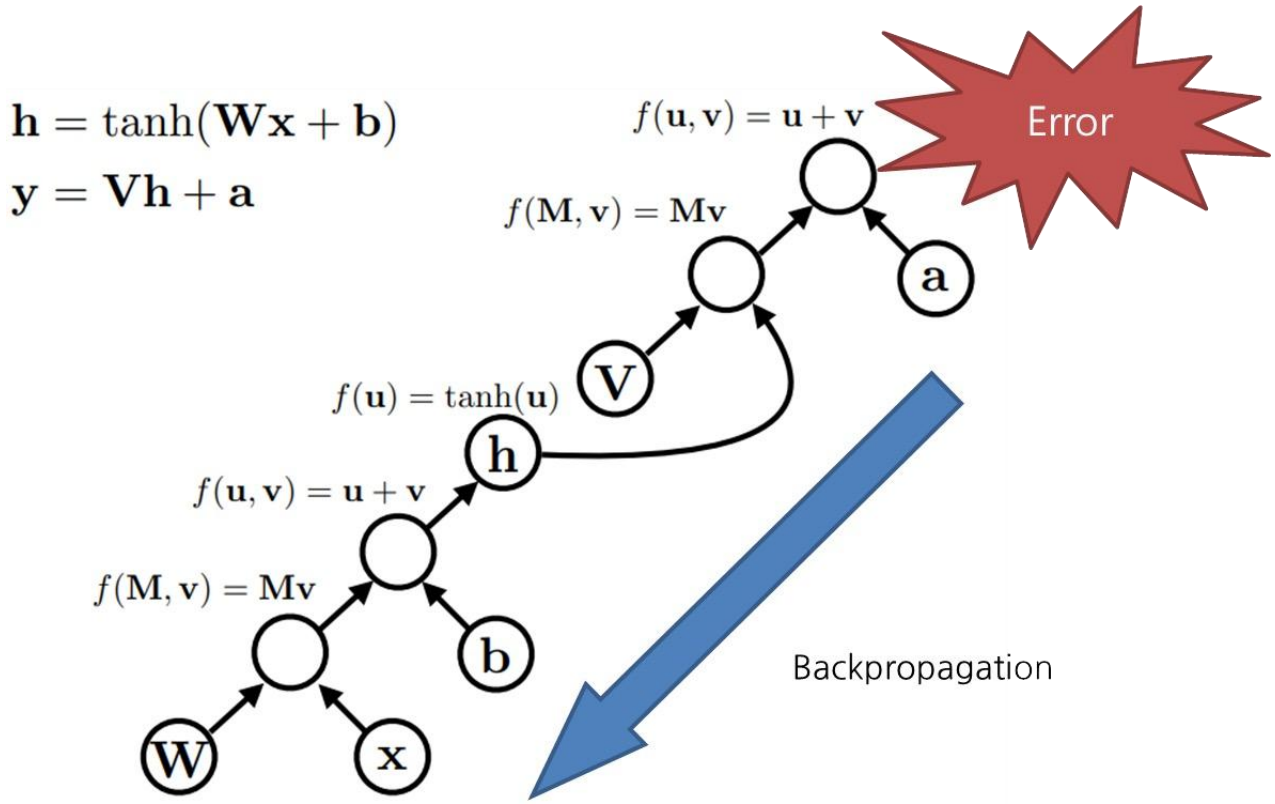
$$\mathbf{y} = \mathbf{V}\mathbf{h} + \mathbf{a}$$



4

Backpropagation

- How to update weights and biases → Backpropagation
  - Calculate error for output layer
  - Calculate contributions to error at each layer and fix it

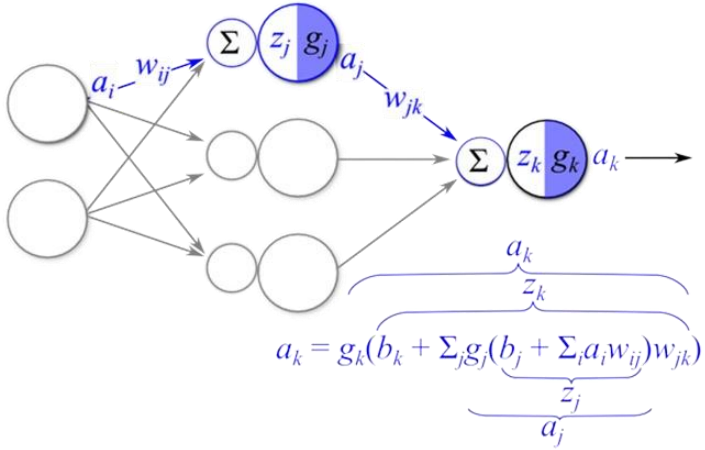


4

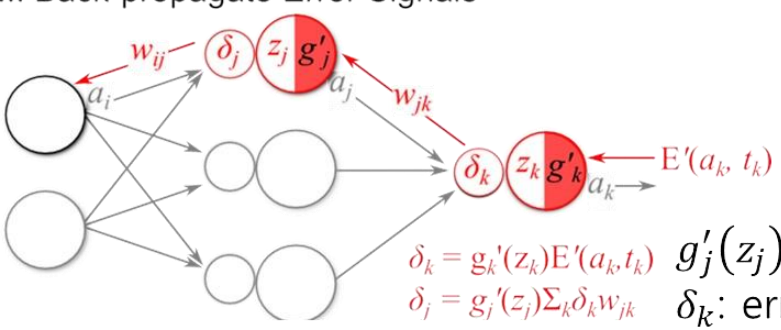
Backpropagation

- How to update weights and biases → Backpropagation

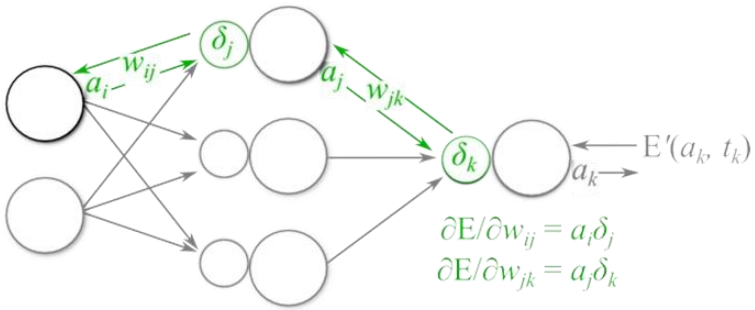
I. Forward-propagate Input Signal



II. Back-propagate Error Signals



III. Calculate Parameter Gradients



IV. Update Parameters

$w_{ij} = w_{ij} - \eta(\partial E/\partial w_{ij})$

$w_{jk} = w_{jk} - \eta(\partial E/\partial w_{jk})$

for learning rate η

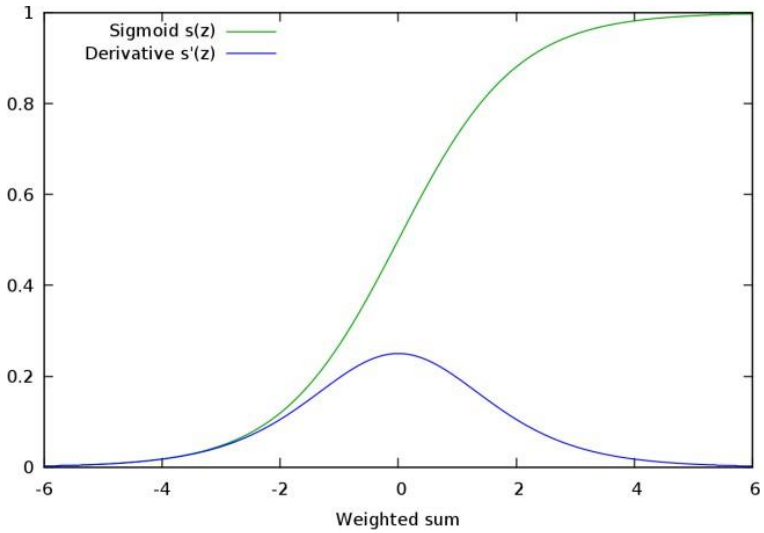
$g'_j(z_j)$: gradient of activation function at layer j

δ_k: error signal at layer k

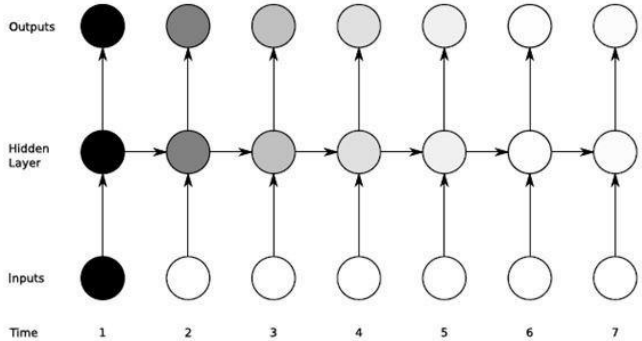
4

Backpropagation

- Activation function
  - Sigmoid function
  - $g(z) = \frac{1}{(1+e^{,z})}$



- Vanishing gradient
  - As more layers using certain activation functions are added to neural networks, **the gradients of the loss function approaches zero, making the network hard to train**.

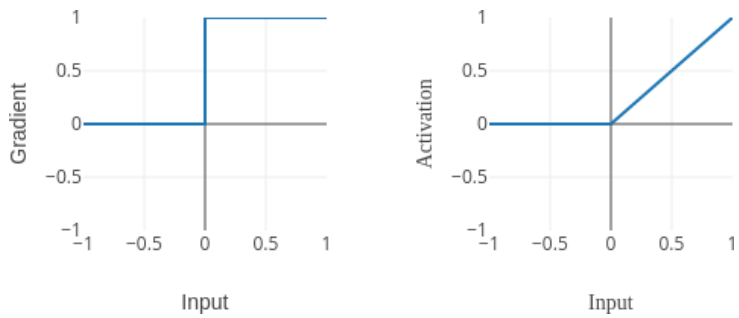


$$a_h^t = \sum_{i=1}^{I} w_{ih} x_i^t + \sum_{h'=1}^{H} w_{h'h} b_{h'}^{t-1}$$

$$b_h^t = \theta_h(a_h^t)$$

Backpropagation

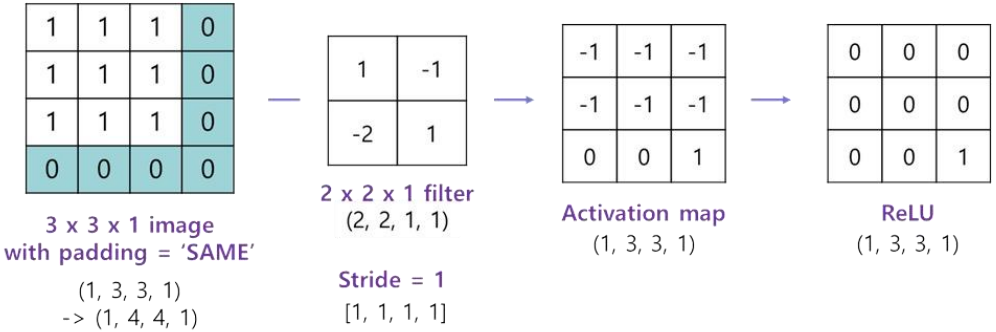
- Vanishing gradient
  - Using ReLU (Rectified Linear Unit)
  - Derivative of ReLU = 1



  - Dying ReLU

    High learning rate, large negative bias, problems in parameter initialization

    Leaky ReLU, PReLU, …



4

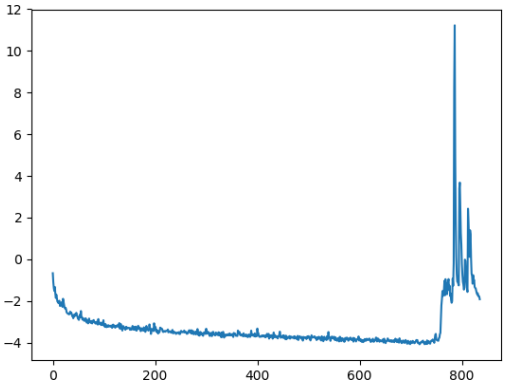
Optimization

- In gradient descent algorithm
  1. **Initialize trainable parameters (weight, bias)**
  2. Calculate the cost and its gradient at given parameters
  3. Update parameters with the gradient and learning rate

- Exploding and vanishing gradient problems
  - Too-large initialization → exploding gradients

    Gradients of cost are too big → cost oscillation
  - Too-small initialization → vanishing gradients

    Early convergence of cost before reaching minimum

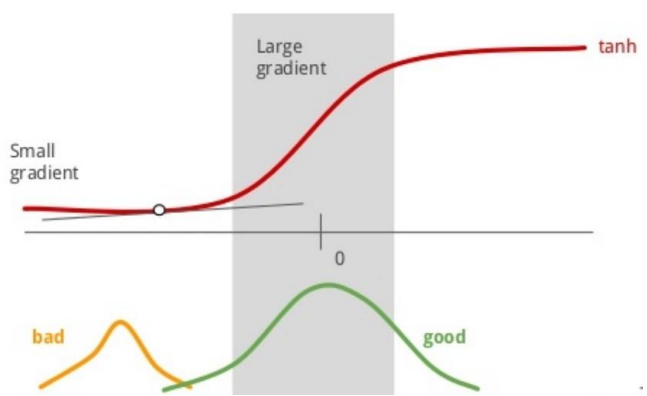


References: https://www.deeplearning.ai/ai-notes/initialization/

4

Optimization

- Initialization methods
  - Avoid cell saturation by setting small parameters randomly
  - The *mean* of the activations should be zero.
  - The *variance* of the activations should stay the same across every layer.



(a) For RELU activation function: This heuristic is called **He-et-al Initialization**.

In this heuristic, we multiplied the randomly generated values of W by:

$$\sqrt{\frac{2}{size^{[l-1]}}}$$

$$W^{[l]} = np.random.randn(size_l, size_l\text{-}1) * np.sqrt(2/size_l\text{-}1)$$

Image Source: link

(b) For tanh activation function: This heuristic is known as **Xavier initialization**.

In this heuristic, we multiplied the randomly generated values of W by:

$$\sqrt{\frac{1}{size^{[l-1]}}}$$

$$W^{[l]} = np.random.randn(size_l, size_l\text{-}1) * np.sqrt(1/size_l\text{-}1)$$

Image Source: link

(c) Another commonly used heuristic is:

$$\sqrt{\frac{2}{size^{[l-1]} + size^{[l]}}}$$

$$W^{[l]} = np.random.randn(size_l, size_l\text{-}1) * np.sqrt(2 / (size_l\text{-}1 + size_l))$$

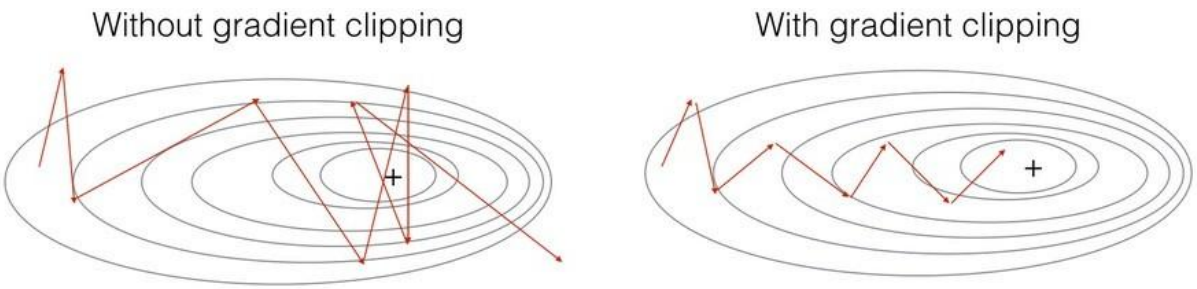
https://www.analyticsvidhya.com/blog/2021/05/how-to-initialize-weights-in-neural-networks/

Narkhede, Meenal V., Prashant P. Bartakke, and Mukul S. Sutaone. "A review on weight initialization strategies for neural networks." *Artificial intelligence review* (2021): 1-32.

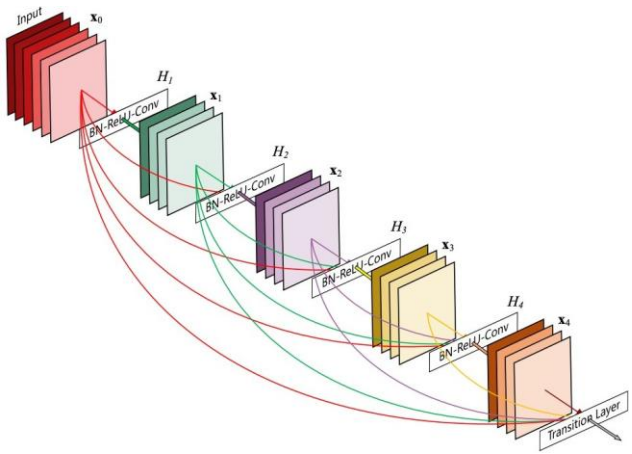
4

Optimization

- Dealing with exploding gradient
  - Gradient clipping



- Dealing with vanishing gradient
  - Activation function: ReLU
  - Orthogonal initialization
  - Skip connections

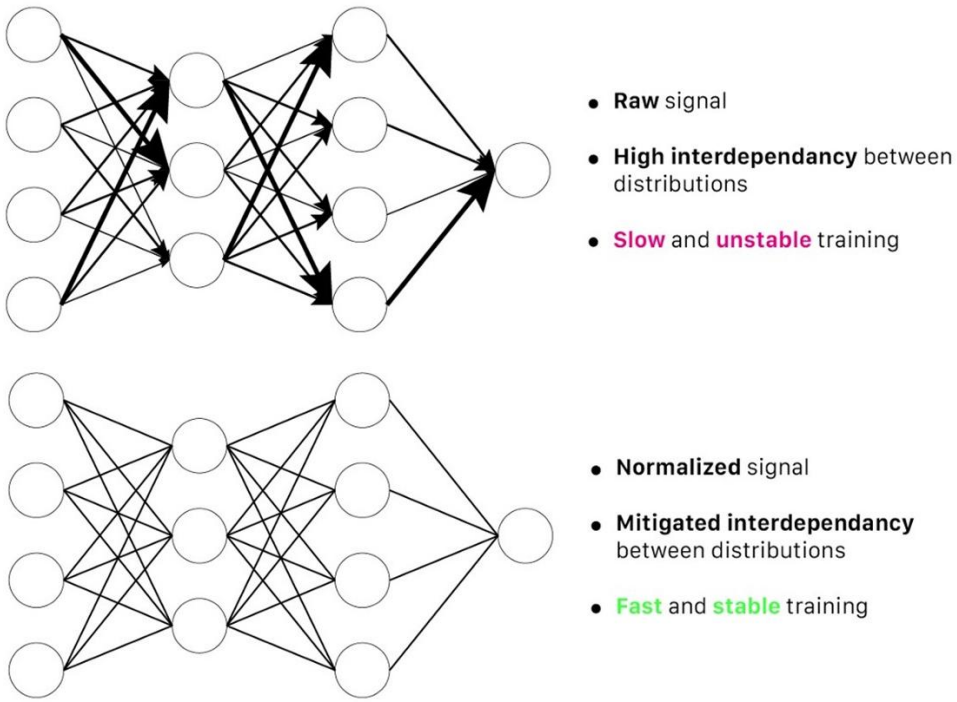


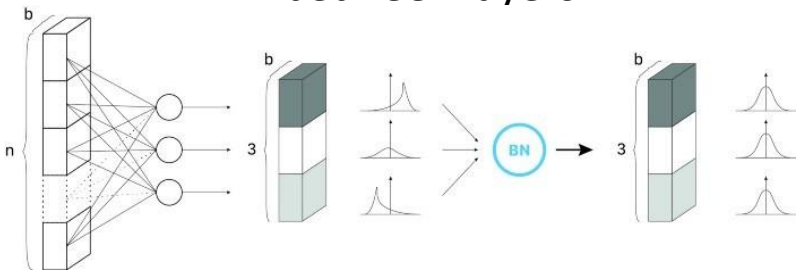
4

DL Techniques for Better Models

- Batch normalization: a solution to slow learning
  - **Normalizing activation vectors from hidden layers** using the first and the second statistical moments (mean and variance) of the batch
  - Prevent cell saturation and covariate shift (...)

    Shifting of distribution between training and testing set
  - Reduce interdependency between layers





https://towardsdatascience.com/batch-normalization-in-3-levels-of-understanding-14c2da90a338

DL Techniques for Better Models

- Normalization Methods

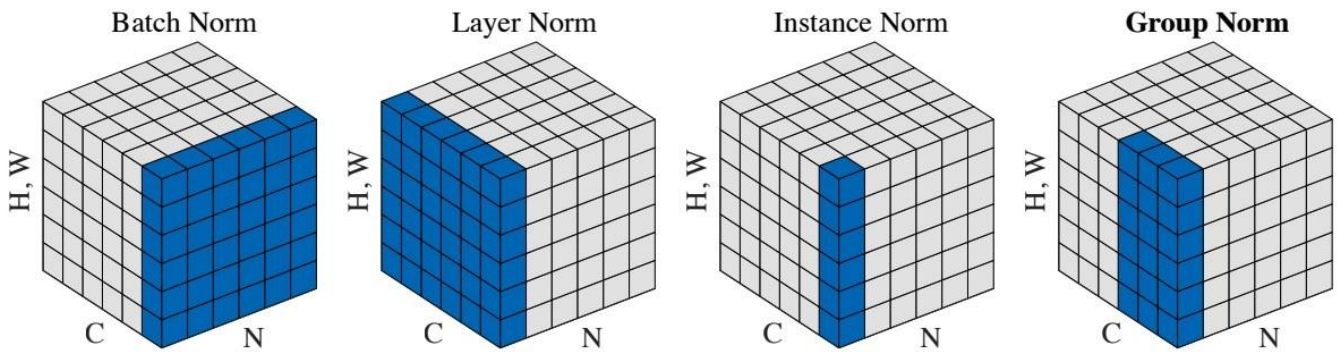
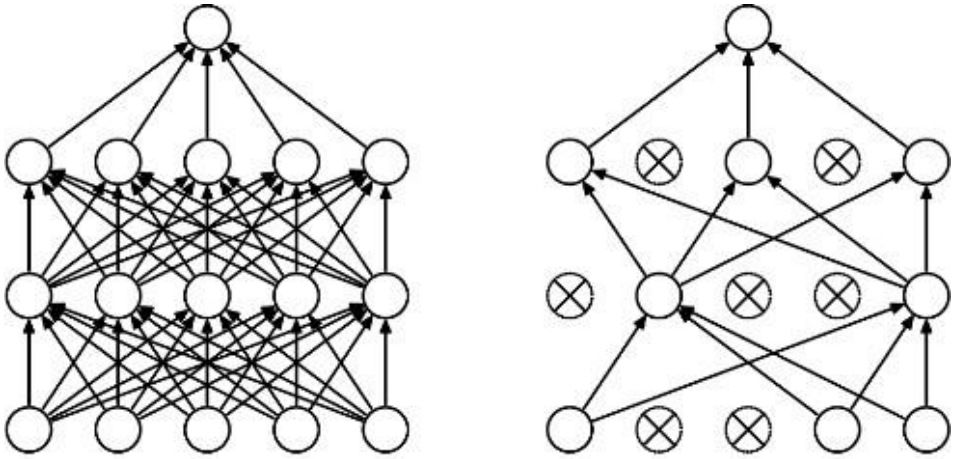


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

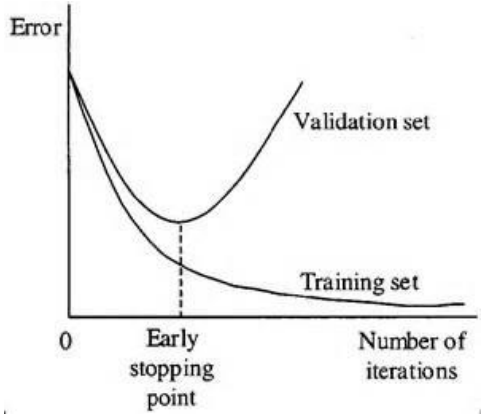
4

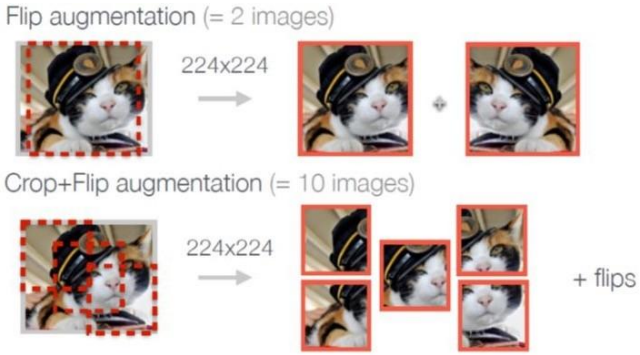
DL Techniques for Better Models

- Solutions to overfitting
  - L1 and L2 regularization
  - Dropout
    - Randomly dropping out hidden nodes
    - Ensemble effect of smaller model
    - Work as regularization
  - Data augmentation
    - Geometric and photometric changes
    - Adding noise
    - Cropping
  - Early stopping
  - Cross validation



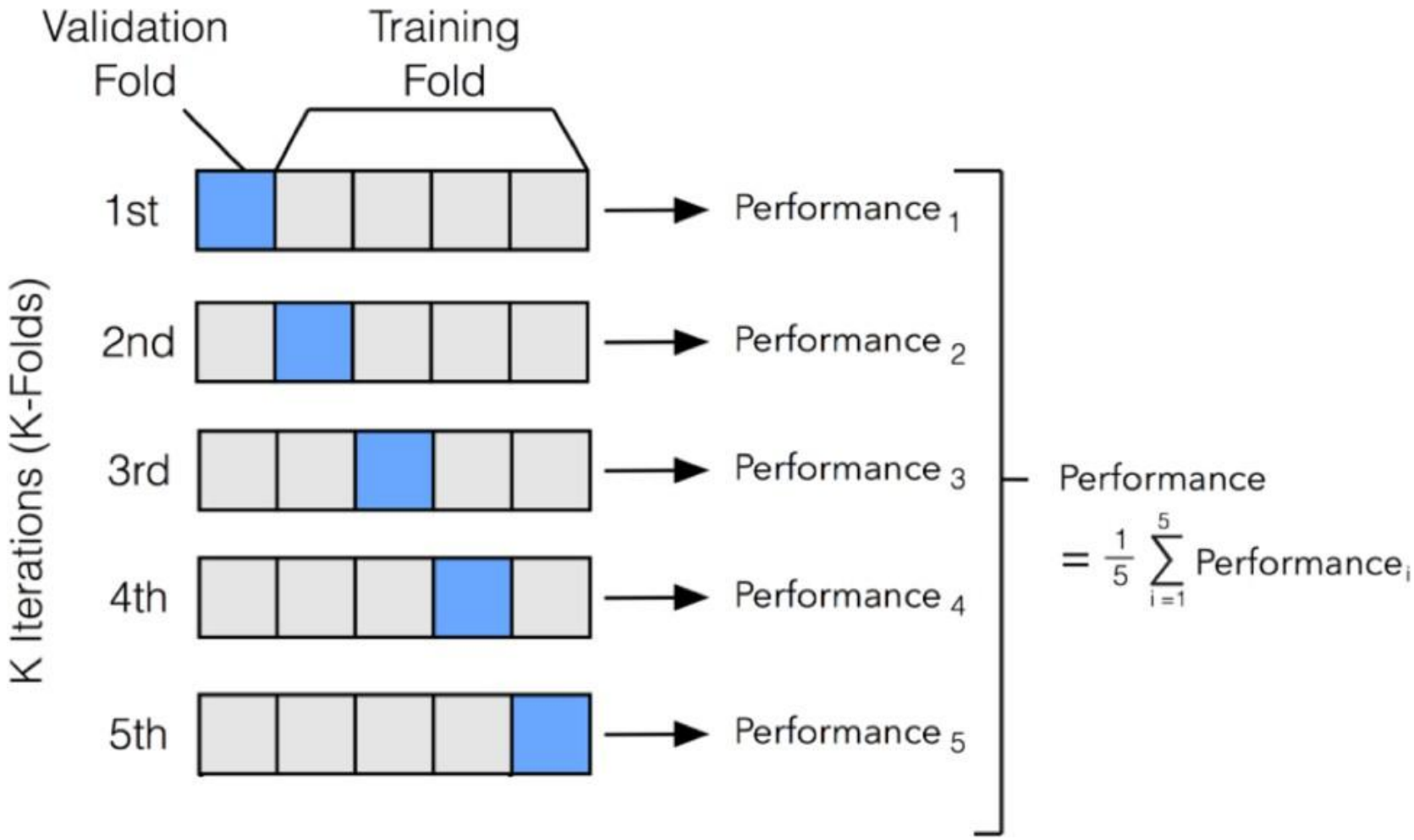
(a) Standard Neural Net (b) After applying dropout.





4

Cross Validation



4

Break

5

PyTorch

- Tutorial
  - https://pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html
  - Tensors
  - DataLoaders
  - Transforms
  - Build neural networks
  - Parameter optimization

A Ω
Our Family UCU Our Pride